ESTRUCTURAS DE DATOS AVANZADAS

Augusto Mauch

ALGORITMIA

Curso 2023/2024

Índice

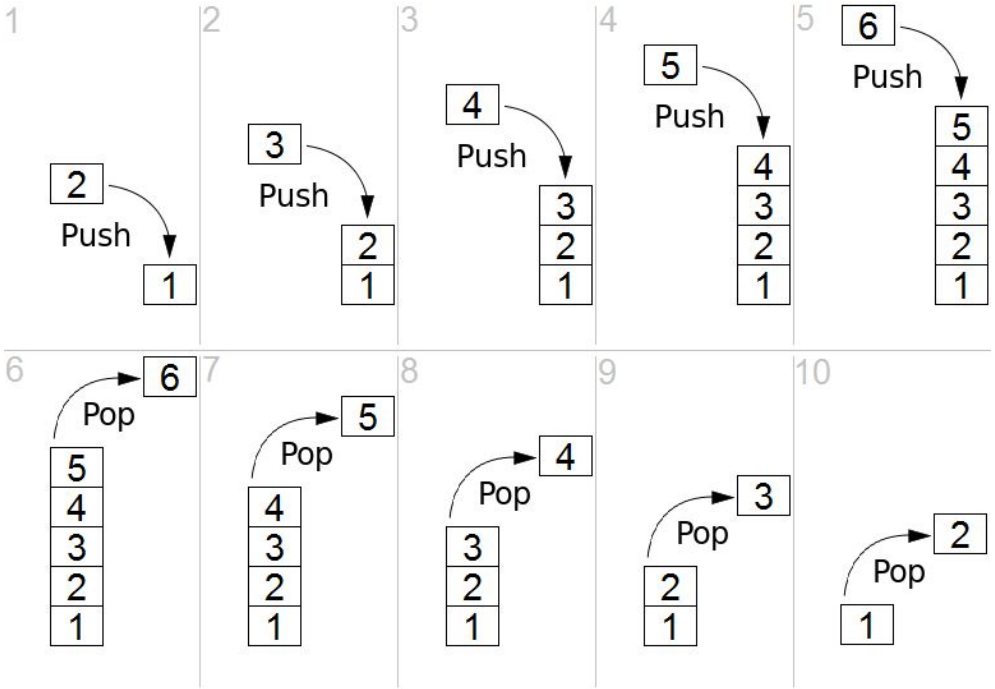
- Repaso. Estructuras de datos básicas
- Grafos
  - Introducción
  - Implementación de grafos
- Árboles
  - Recorridos en anchura y profundidad
  - Árboles binarios de búsqueda
  - Montículos

Pilas

- Estructura LIFO (Last In First Out, el último añadido es el primero que sale)

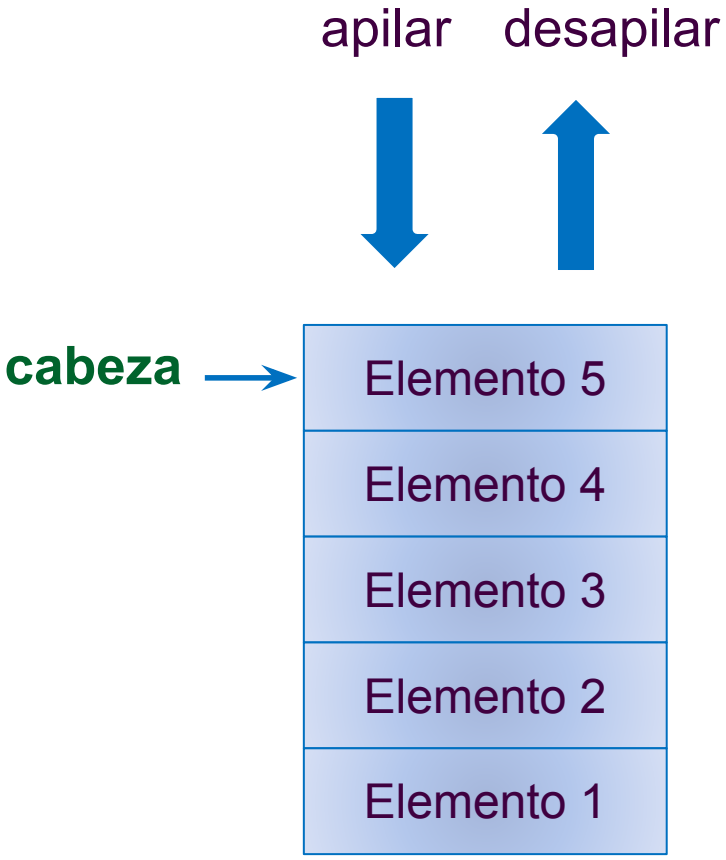






Pilas

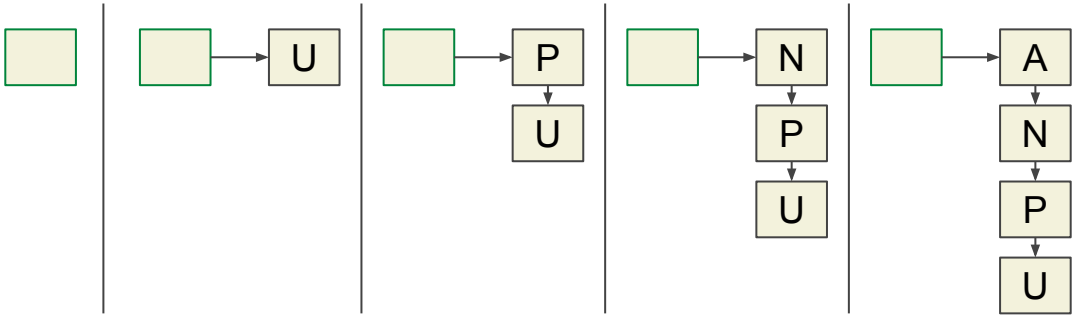
- Apilar
  - Añade un elemento en la cabeza de la pila
- Desapilar
  - Elimina un elemento de la cabeza de la pila, falla si la pila está vacía
- Cima
  - Muestra el elemento en la cabeza de la pila, falla si la pila está vacía
- ¿Está vacía?
  - Comprueba si la pila contiene elementos o no
- Vaciar
  - Eliminan uno a uno todos los elementos de la pila



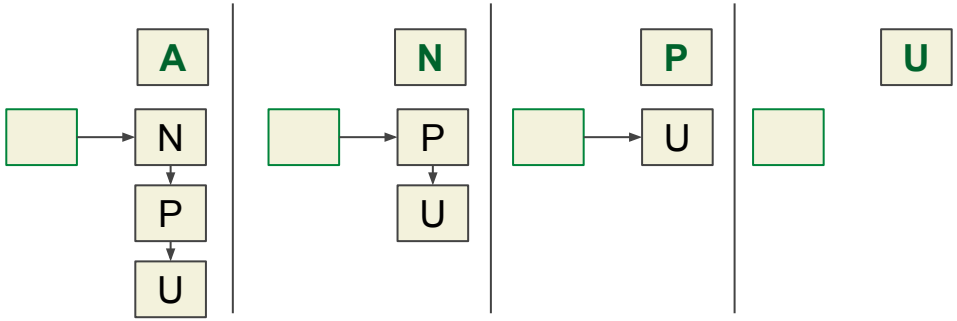
Pilas - Aplicaciones

Leer una secuencia de caracteres e imprimirla al revés (UPNA)

- Introducir los elementos en la pila



- Imprimir los elementos conforme los sacamos de la pila



Pilas - Aplicaciones

Comprobar si una cadena de caracteres tiene paréntesis y corchetes balanceados

([ab]c[de]) -> BALANCEADOS

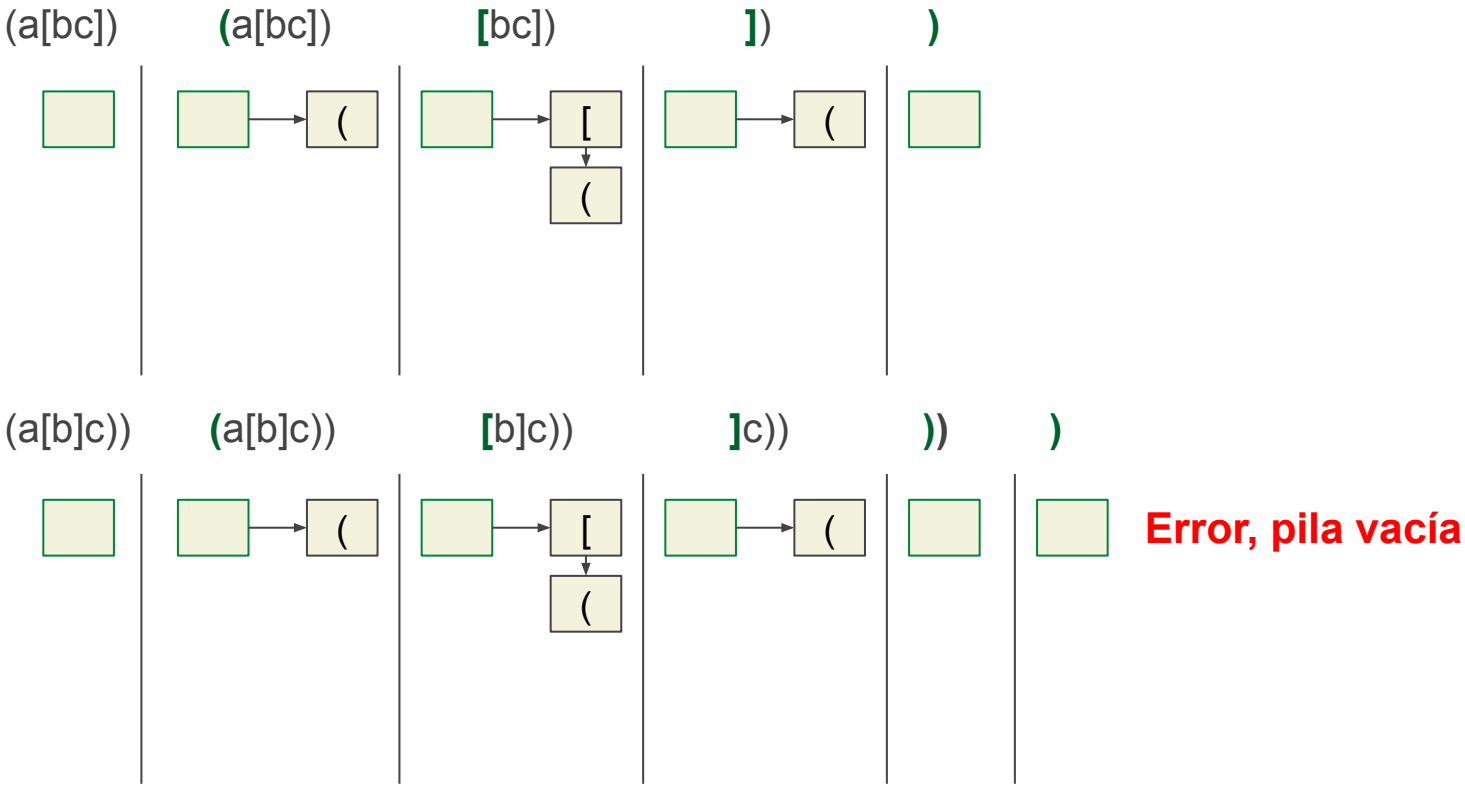
([ab]]c), ([ab] -> NO BALANCEADOS

- Recorrer cadena de caracteres
  - Parentesis/corchete de apertura -> meterlo en la pila
  - Paréntesis/corchete de cierre -> comprobar la cima de la pila. Si no se corresponde con el símbolo de apertura correspondiente, salir con error. Si la pila está vacía, salir con error

- Al terminar, si la pila no está vacía, salir con error (paréntesis/corchete de apertura sin símbolo de cierre correspondiente)

Pilas - Aplicaciones

Comprobar si una cadena de caracteres tiene paréntesis balanceados - ejemplos



Visto en openbravo

- Stack trace - Pila de llamadas
  - Estructura que guarda información sobre la subrutina que se está ejecutando en cada momento

```
if ((isNewReceipt || isModified) && !doCancelLayaway) {
  verifyOrderLineTax(jsonorder);
  lineReferences = new ArrayList<OrderLine>();
  createOrderAndLines(jsonorder, order, orderlines, lineReferences);
}
```

```
OrderLoader.saveRecord(JSONObject) line: 345
OrderLoader(DataSynchronizationProcess).saveRecord(JSONArray, boolean) line: 204
OrderLoader(DataSynchronizationProcess).exec(JSONObject, boolean) line: 156
```

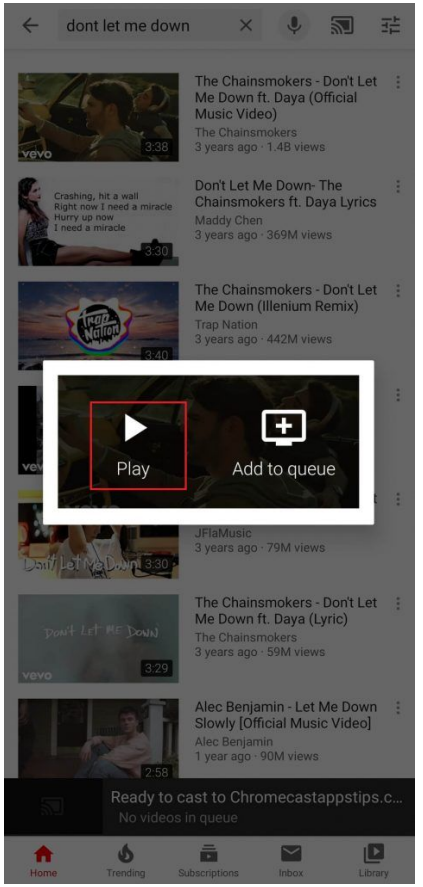
```
void createOrderAndLines(final JSONObject jsonorder, final Order order,
    final JSONArray orderlines, final ArrayList<OrderLine> lineReferences)
  createOrder(order, jsonorder);
  OBDal.getInstance().save(order);
```

```
OrderLoader.createOrderAndLines(JSONObject, Order, JSONArray, ArrayList<OrderLine>) line: 1119
OrderLoader.saveRecord(JSONObject) line: 345
OrderLoader(DataSynchronizationProcess).saveRecord(JSONArray, boolean) line: 204
OrderLoader(DataSynchronizationProcess).exec(JSONObject, boolean) line: 156
```

Colas

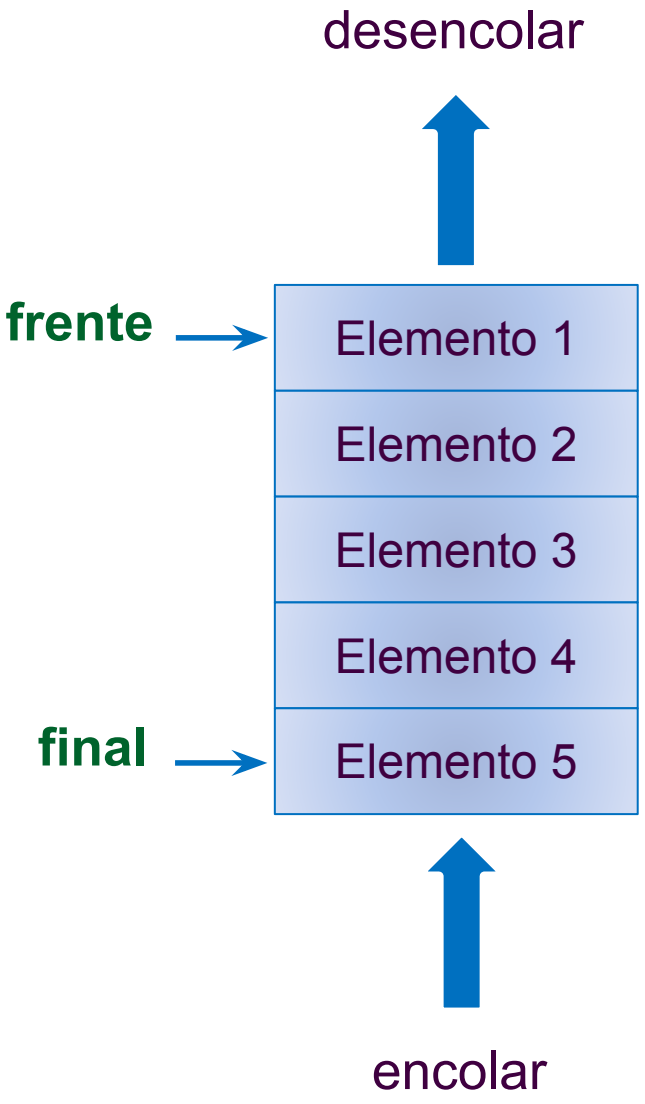
- Estructura FIFO (First In First Out, el primero que entra es el primero que sale)





Colas

- Encolar
  - Añade un elemento en el final de la cola
- Desencolar
  - Elimina el elemento del frente de la cola
- Frente
  - Muestra el elemento del frente de la cola
- ¿Está vacía?
  - Comprueba si la cola contiene elementos o no
- Vaciar
  - Elimina uno a uno todos los elementos de la cola



Visto en openbravo

Punto de venta



Servidor



Base de datos



Eventos

1. Alta de cliente
2. Compra de camiseta
3. Devolución de camiseta

Pila

Cola

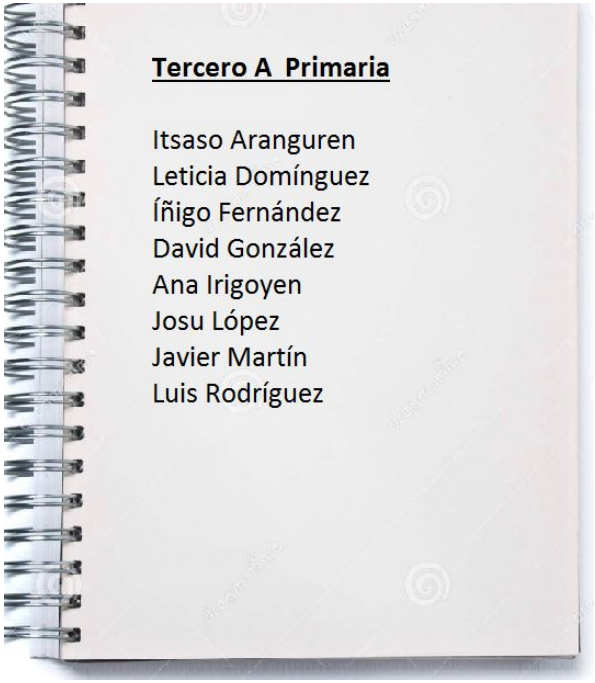
Orden de procesamiento

3. Devolución de camiseta
2. Compra de camiseta
1. Alta de cliente

1. Alta de cliente
2. Compra de camiseta
3. Devolución de camiseta

Listas


SHOPPING LIST
Leche
Aceite
Garbanzos
Salmón
Yogures

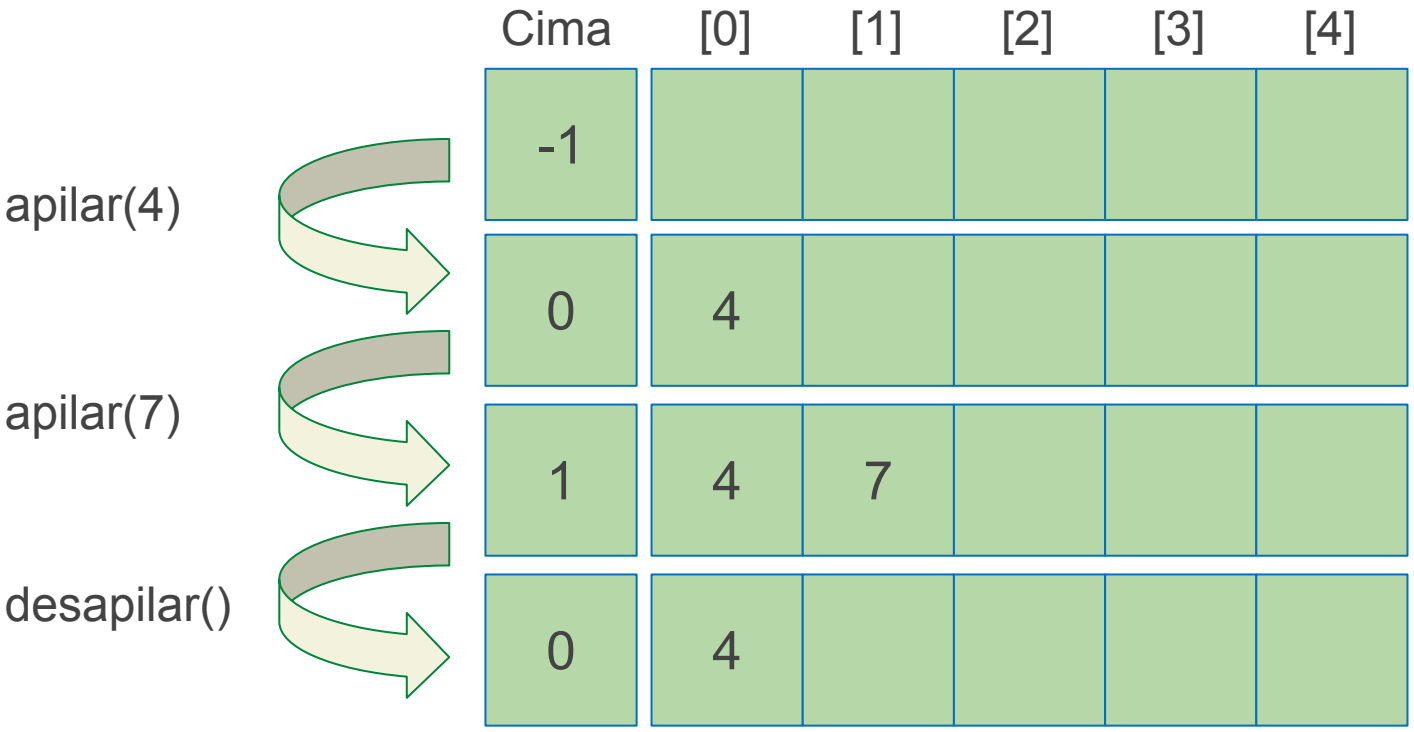

Tercero A Primaria
Itsaso Aranguren
Leticia Domínguez
Íñigo Fernández
David González
Ana Irigoyen
Josu López
Javier Martín
Luis Rodríguez

Listas

- Insertar
  - Añade un elemento a la lista en una posición dada
- Obtener
  - Lee un elemento en una posición dada
- Eliminar
  - Elimina un elemento de la lista. Puede ser por valor o por posición
- ¿Está vacía?
  - Comprueba si la lista contiene elementos o no

Implementación de estructuras lineales

- Se puede reservar un array con X posiciones adyacentes
- Ejemplo: Pila de como máximo 5 elementos

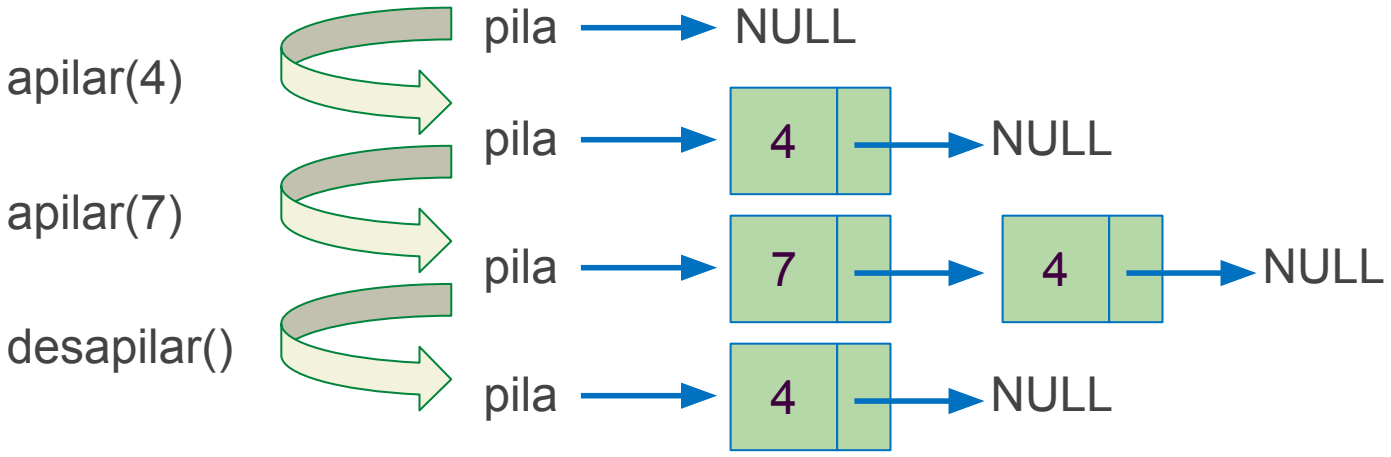


Implementación de estructuras lineales

- ¿Y si no se cuántos elementos va a tener mi pila?
- Reserva de un vector muy grande -> ineficiente
- Ideal: que la pila tenga en cada momento el tamaño ajustado a los elementos que contiene, estructura dinámica

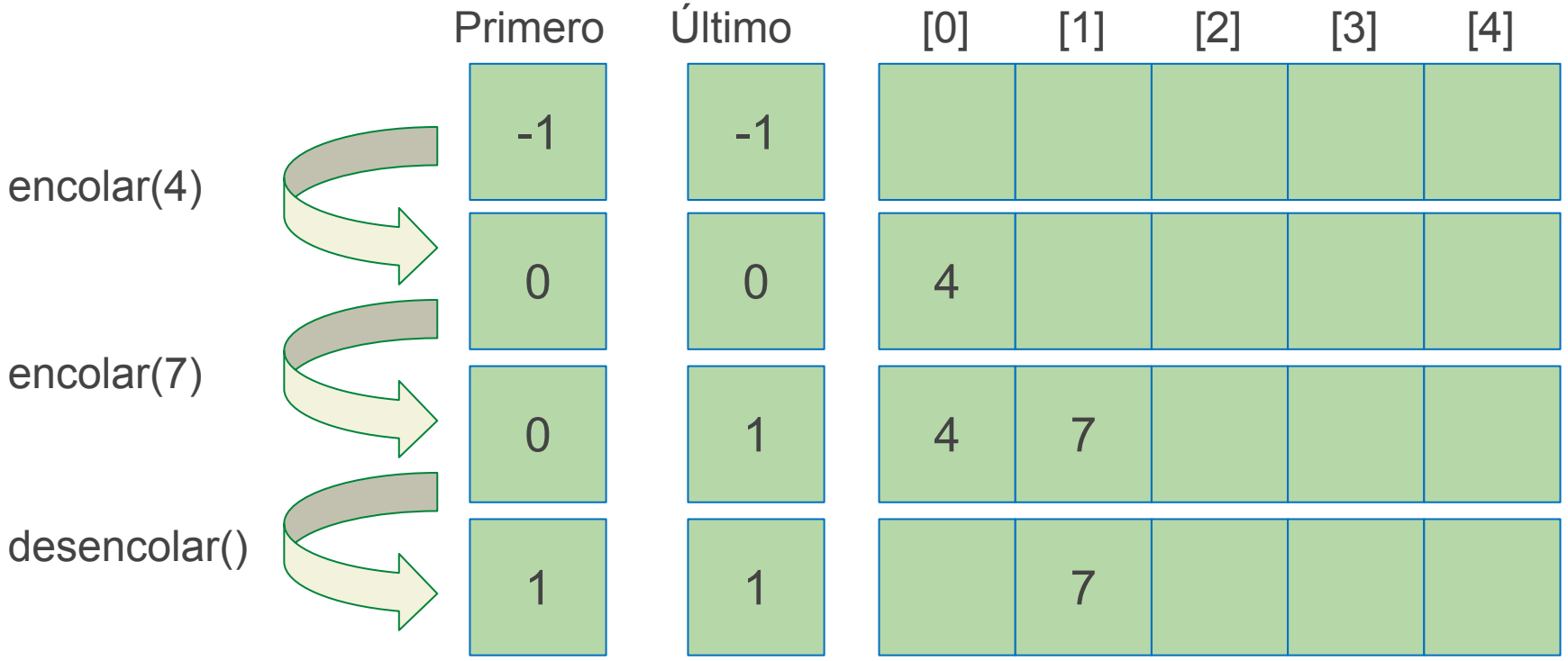
Implementación de estructuras lineales

- La variable pila es un puntero a una celda
- Si la pila está vacía, apunta a NULL
- Cada celda contiene espacio para un elemento, y un puntero que apunta a la siguiente celda de la pila
- El puntero de la última celda apunta a NULL



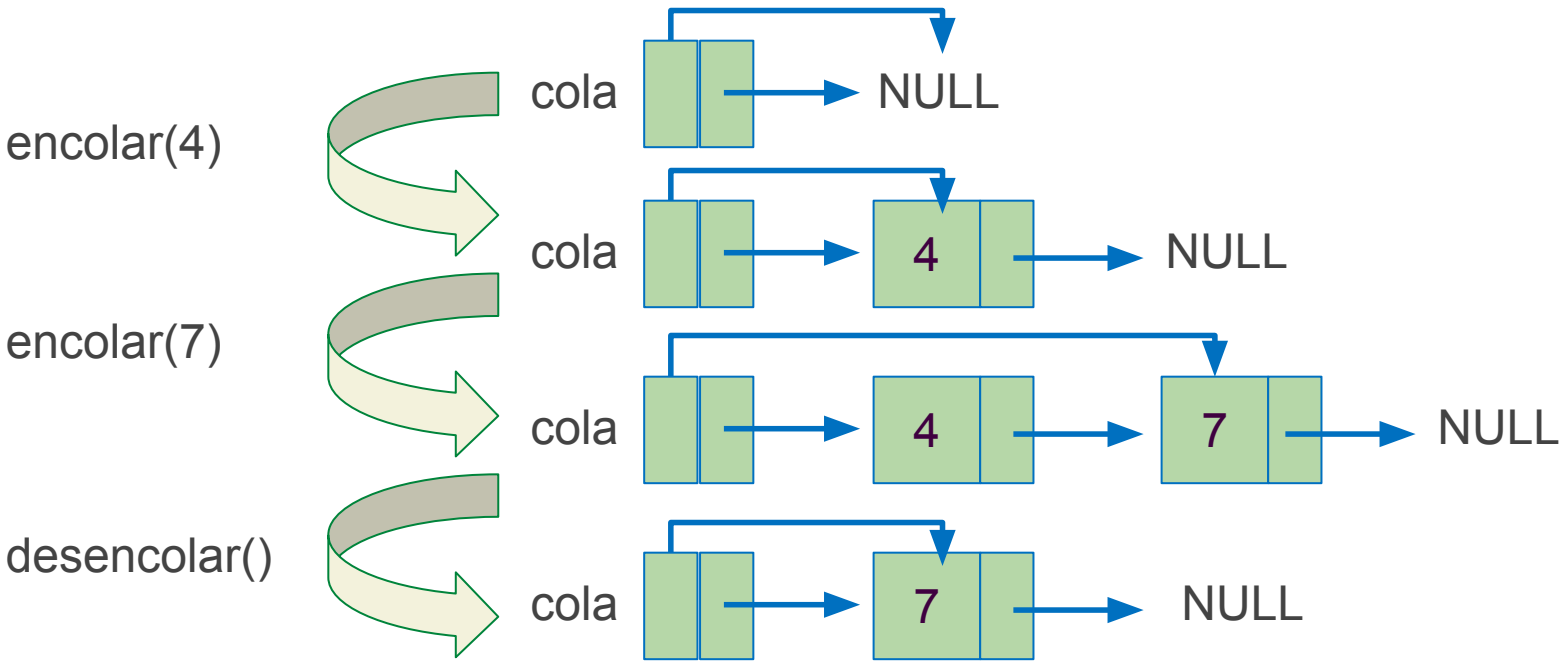
Implementación de estructuras lineales

- Implementación estática de una cola
- Ahora necesitamos dos índices: primero, último



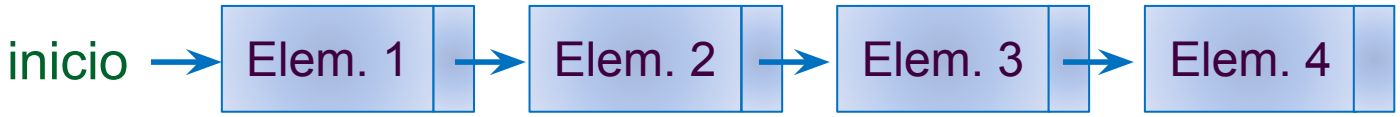
Implementación de estructuras lineales

- Implementación dinámica de una cola
- Ahora utilizamos dos punteros

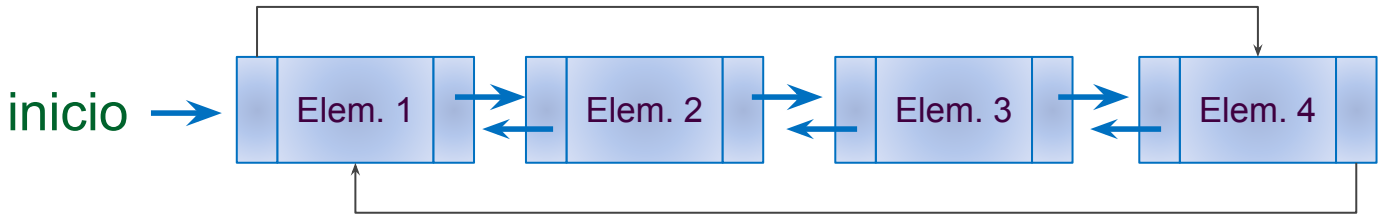


Listas

- Diferentes implementaciones en función de requisitos
- Lista simple
  - A partir de cada elemento se puede acceder al siguiente



- Lista circular doblemente enlazada
  - A partir de cada elemento se puede acceder al siguiente y al anterior. El último apunta al primero, el primero apunta al último



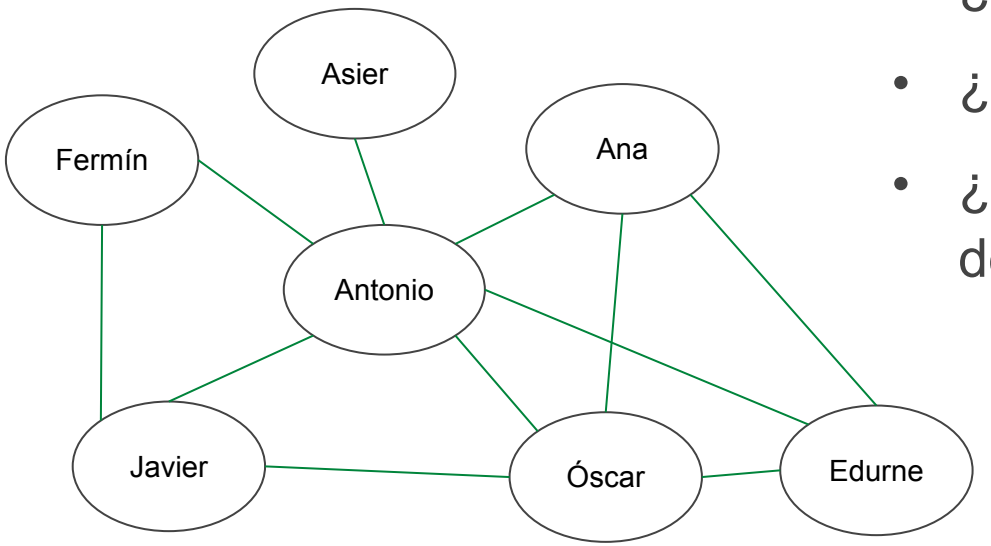
Índice

- Repaso. Estructuras de datos básicas
- Grafos
  - Introducción
  - Implementación de grafos
- Árboles
  - Recorridos en anchura y profundidad
  - Árboles binarios de búsqueda
  - Montículos

Grafos - Introducción

Estructura de datos que permite modelar relaciones entre elementos.

Ejemplo: grafo de amistad en redes sociales



- ¿Óscar y Fermín son amigos?
- ¿Quién tiene más amigos?
- ¿Cual es la manera más rápida de llegar de Fermín a Edurne?

Grafos - Introducción

Otro ejemplo, origen de la teoría de grafos: Problema de los puentes de Königsberg

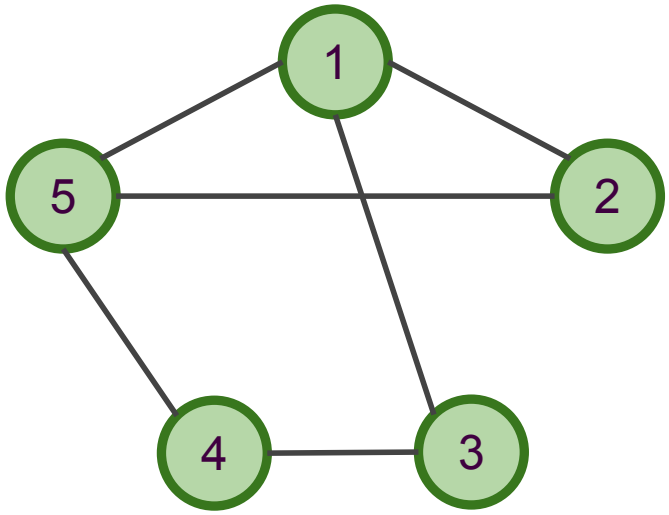
4 regiones, unidas por 7 puentes

¿Es posible partir de una región, recorrer todos los puentes una sola vez y regresar al punto de partida?



Grafos

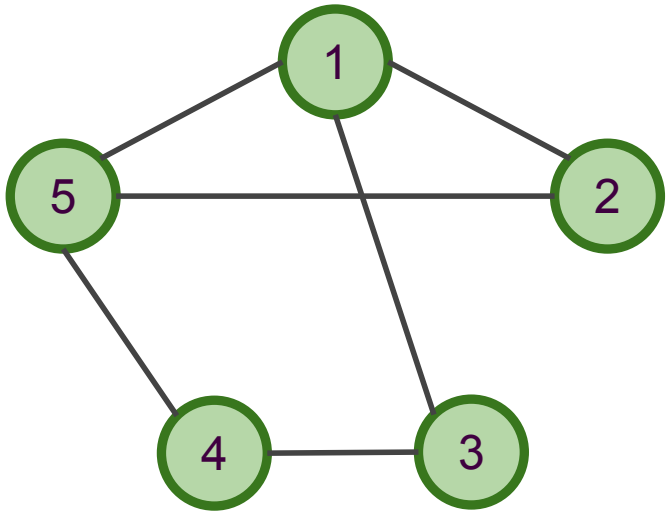
- Un grafo **G = ⟨N,A⟩** está formado por un conjunto finito y no vacío de vértices o nodos N y un conjunto A de pares de vértices a los que se llama aristas o arcos



N={1, 2, 3, 4, 5}
A={{1,2}, {1,3}, {1,5},
{2,5}, {3,4}, {4,5}}

Grafos dirigidos y no dirigidos

- En un grafo no dirigido, el par de vértices que representa una arista está desordenado ({u,v} y {v,u} representan la misma arista)



N={1, 2, 3, 4, 5}

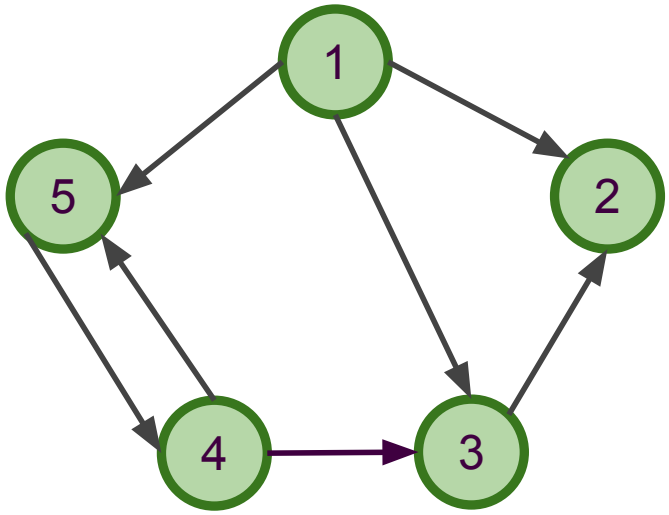
A={{1,2}, {1,3}, {1,5},
{2,5}, {3,4}, {4,5}}

=

A={{1,2}, {1,3}, {5,1},
{5,2}, {4,3}, {5,4}}

Grafos dirigidos y no dirigidos

- En un grafo dirigido las aristas o arcos se representan por pares de vértices ordenados (u,v)

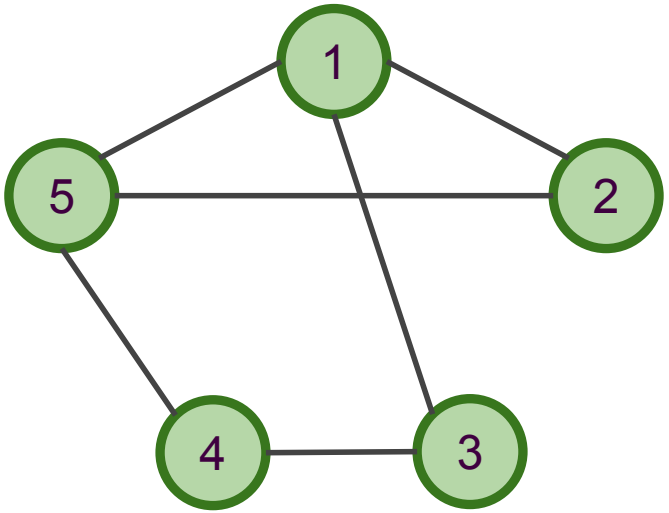


N={1, 2, 3, 4, 5}

A={(1,2), (1,3), (1,5),
(3,2), (4,3), (4,5),
(5,4)}

Grafos

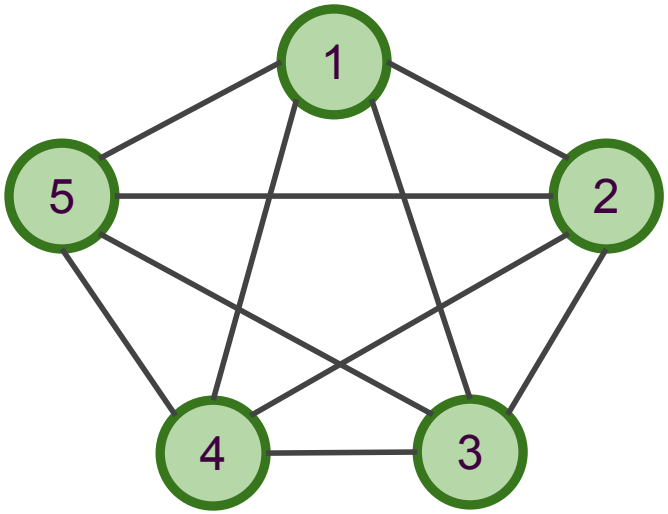
- Dos vértices u, v de un grafo G = ⟨N,A⟩ se dicen **adyacentes** si {u, v} ∈ A
- Dos aristas son **adyacentes** si tienen un vértice en común
- Si a = {u, v} decimos que la arista a es **incidente** a los vértices u y v.
- El **grado** de un vértice es el número de arcos incidentes a él. El grado de un vértice u se denota gr(u).

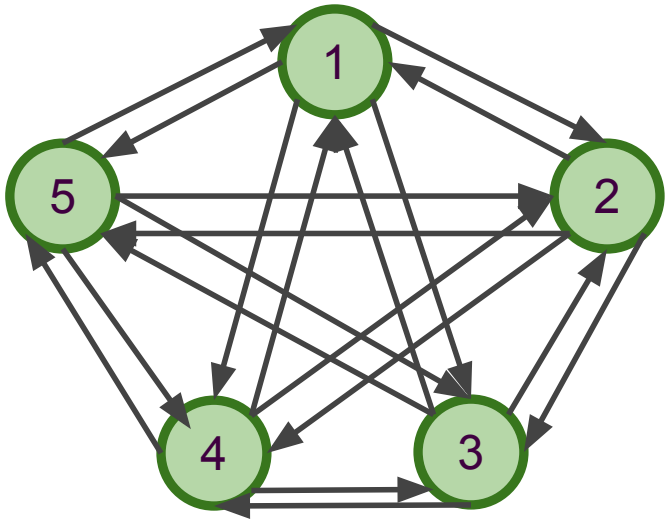


- 1 y 2 son vértices adyacentes porque {1,2}ϵA
- {1,2} y {1,3} son aristas adyacentes
- La arista {1,2} es incidente a los vértices 1 y 2
- gr(1)=3
- gr(4)=2

Grafos

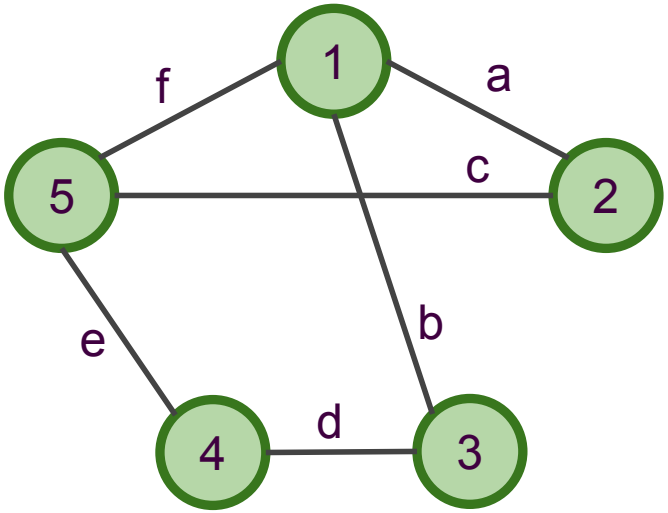
- Un grafo simple G = ⟨N,A⟩ se dice **completo** si cada vértice está conectado a cualquier otro vértice en G
  - Un grafo no dirigido de n vértices es completo si tiene exactamente n(n-1)/2 arcos
  - Un grafo dirigido de n vértices es completo si tiene exactamente n(n-1) arcos



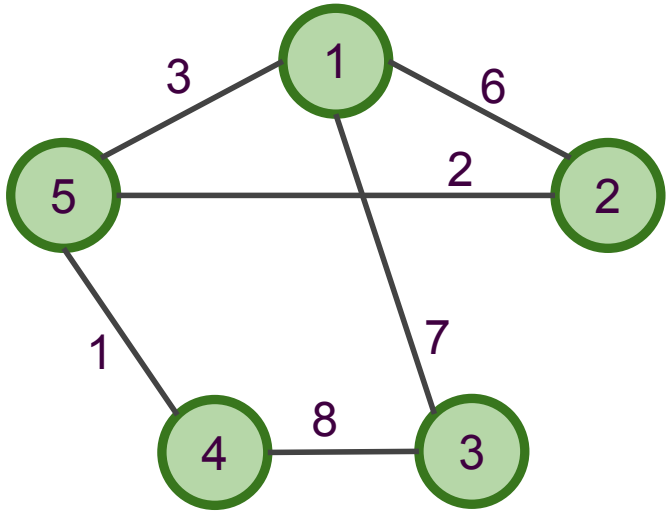


Grafos ponderados

- Un grafo G es un grafo **etiquetado** si sus aristas y/o vértices tienen asignado alguna identificación
- En particular, G es un grafo **ponderado** si a cada arista a de G se le asigna un número no negativo w(a) denominado **peso** o longitud de a



Grafo etiquetado



Grafo ponderado

Índice

- Repaso. Estructuras de datos básicas
- Grafos
  - Introducción a grafos
  - Implementación de grafos
- Árboles
  - Recorridos en anchura y profundidad
  - Árboles binarios de búsqueda
  - Montículos

Implementación de grafos

- Existen diversas estructuras
- Cada una es apropiada en diferentes situaciones

- Estructura estática: Matriz de adyacencia
- Estructura dinámica: Lista de adyacencia

Matriz de adyacencia

- Sea G = ⟨N,A⟩ un grafo con n vértices, n ≥ 1. La matriz de adyacencia D es un array bidimensional n x n definido de la siguiente forma:

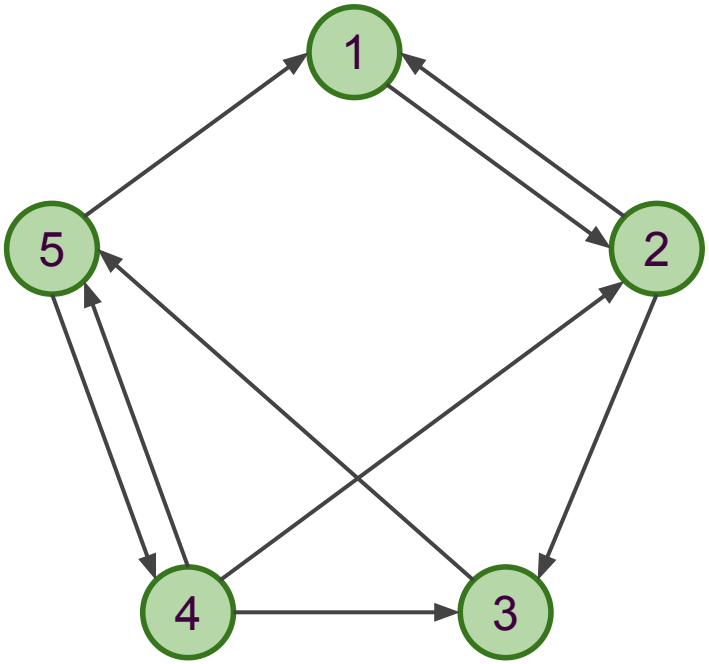
$$D(i,j) = \begin{cases} 1 & si\ \{i,j\} \in A\ \ ((i,j)\ \ en\ grafos\ dirigidos) \\ 0 & en\ caso\ contrario \end{cases}$$

- Si G es un grafo ponderado, la matriz de adyacencia se define de la siguiente forma:

$$D(i,j) = \begin{cases} w(i,j) & si\ \{i,j\} \in A\ \ ((i,j)\ \ en\ grafos\ dirigidos) \\ cte & en\ caso\ contrario \end{cases}$$

Donde cte es un valor constante (0, inf, -inf, …)

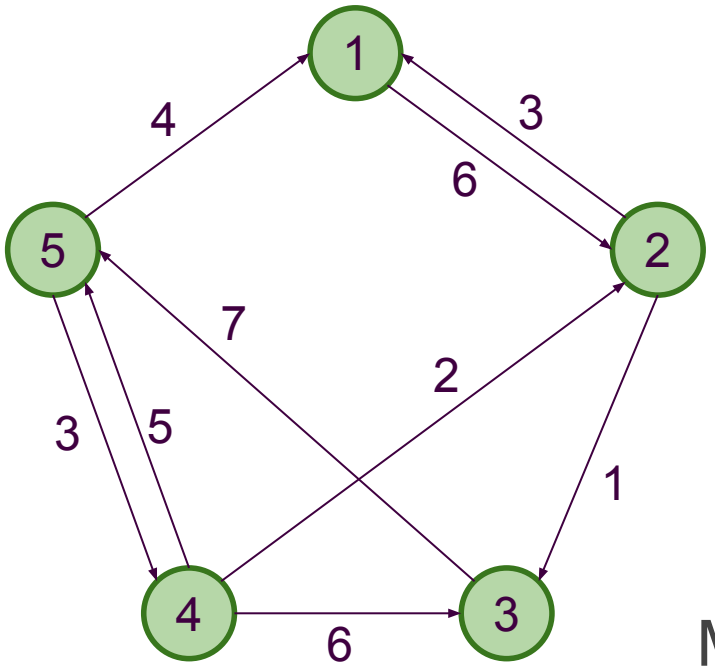
Matriz de adyacencia



$$M = \begin{pmatrix} 0 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 0 & 1 & 0 \end{pmatrix}$$

M(1,4) = 0, no existe arista (1, 4)
M(3,5) = 1, existe arista (3, 5)

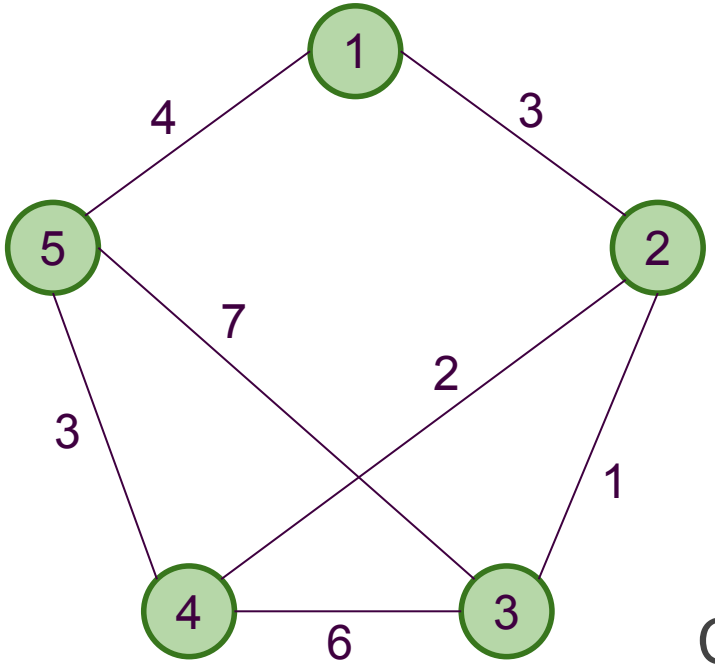
Matriz de adyacencia



$$M = \begin{pmatrix} 0 & 6 & 0 & 0 & 0 \\ 3 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 7 \\ 0 & 2 & 6 & 0 & 5 \\ 4 & 0 & 0 & 3 & 0 \end{pmatrix}$$

M(1,4) = 0, no existe arista (1, 4)
M(3,5) = 7, existe arista (3, 5) con peso 7

Matriz de adyacencia



$$M = \begin{pmatrix} 0 & 3 & 0 & 0 & 4 \\ 3 & 0 & 1 & 2 & 0 \\ 0 & 1 & 0 & 6 & 7 \\ 0 & 2 & 6 & 0 & 3 \\ 4 & 0 & 7 & 3 & 0 \end{pmatrix}$$

Como el grafo no es dirigido,
M(i,j) = M(j,i), el grafo es simétrico

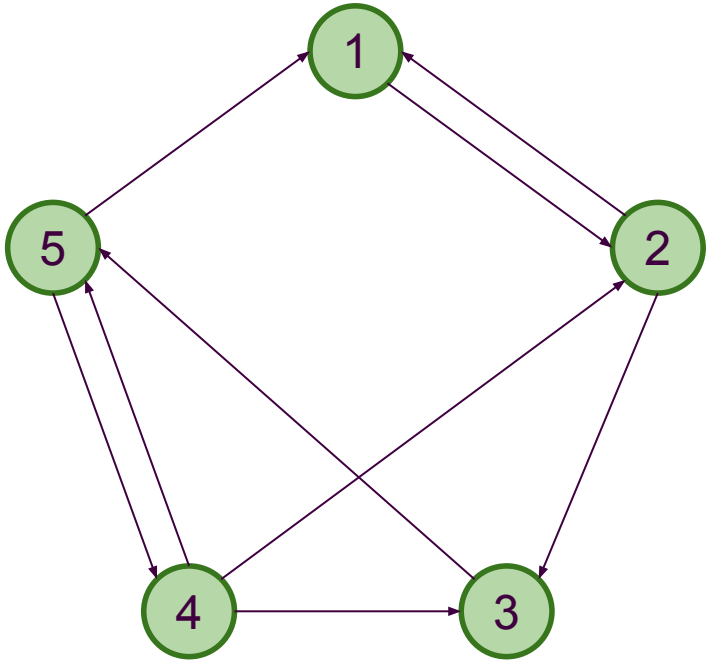
Matriz de adyacencia

- Es inmediato ver si dos vértices i, j están conectados, consultando M(i,j). El orden de eficiencia de esta operación es independiente del número de vértices y arcos
- No es inmediato ver cuántos arcos tiene el grafo
- Se utiliza mucha memoria extra en grafos poco densos (gran número de vértices y pocas aristas)

Lista de adyacencia

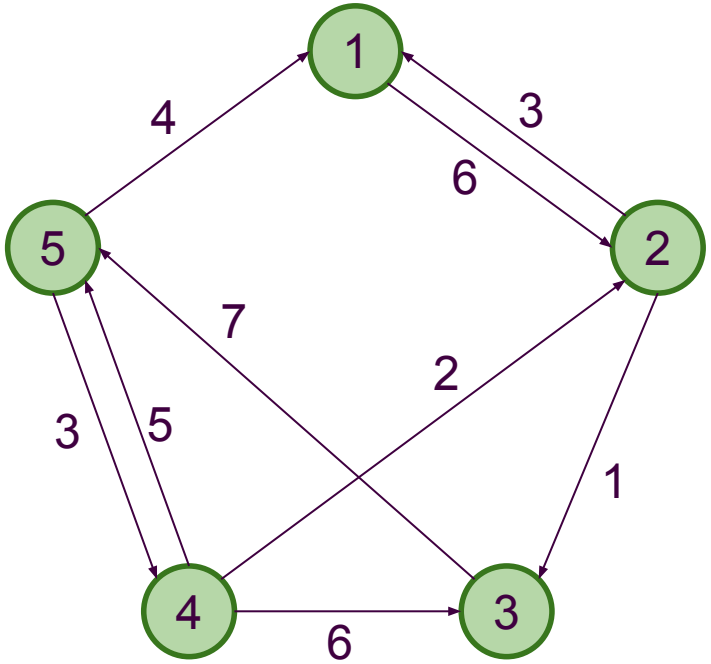
- Sea G =⟨N,A⟩ un grafo con n vértices, n ≥ 1. La lista de adyacencia es una lista de n listas enlazadas. Cada elemento de la lista está asociado con un vértice i, y representa mediante una lista los nodos destino de las aristas con origen en i.
- Si G es un grafo ponderado, entonces en cada lista se almacenan los nodos destino y los pesos de las aristas con origen en i.

Lista de adyacencia



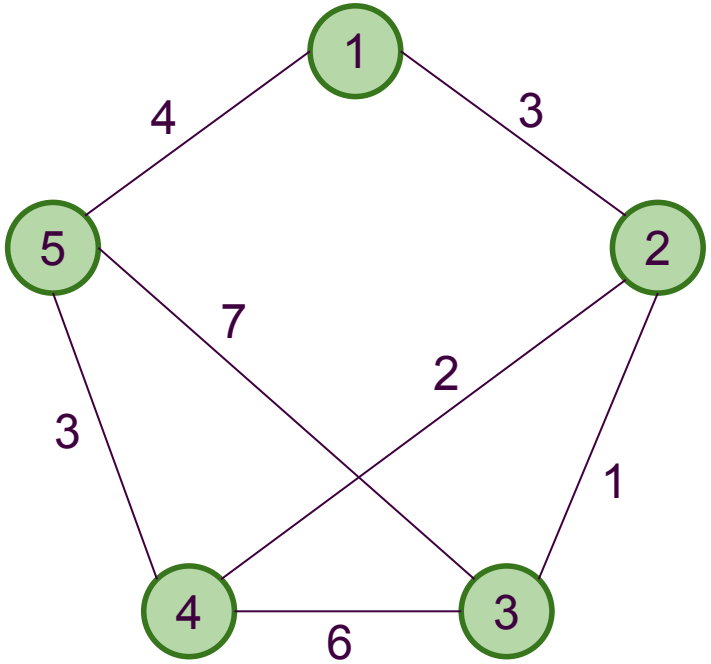
1	2		
2	1	3	
3	5		
4	2	3	5
5	1	4	

Lista de adyacencia



1	(2,6)		
2	(1,3)	(3,1)	
3	(5,7)		
4	(2,2)	(3,6)	(5,5)
5	(1,4)	(4,3)	

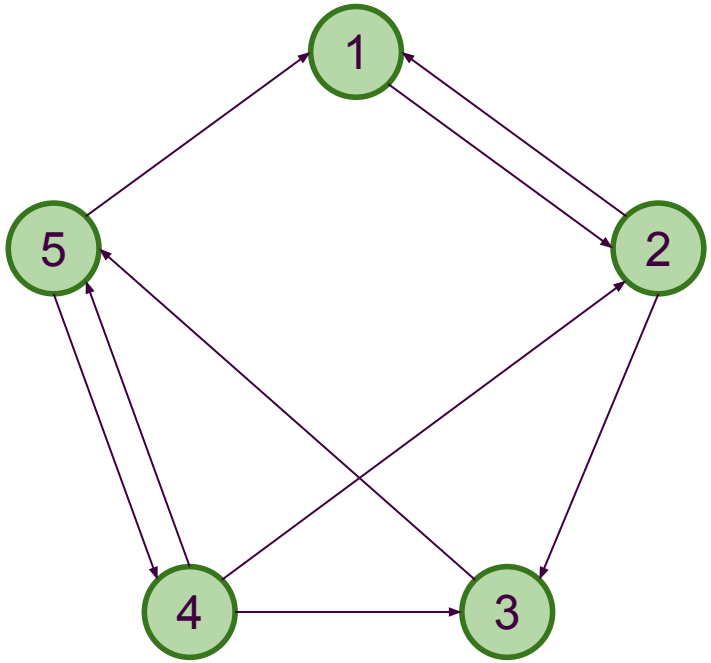
Lista de adyacencia

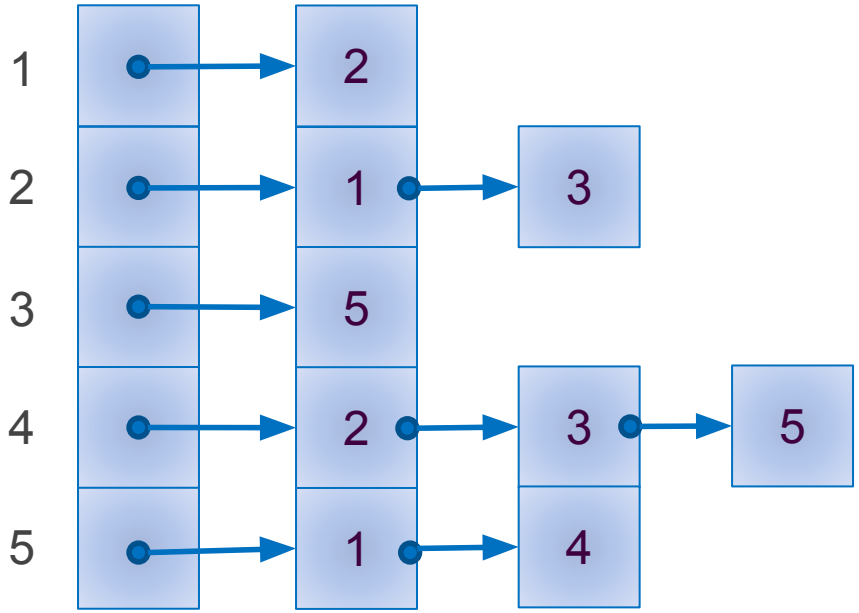


1	(2,3)	(5,4)	
2	(1,3)	(3,1)	(4,2)
3	(2,1)	(4,6)	(5,7)
4	(2,2)	(3,6)	(5,3)
5	(1,4)	(3,7)	(4,3)

Lista de adyacencia

- No podemos implementar una matriz con diferente número de columnas en cada fila
- Utilizamos punteros



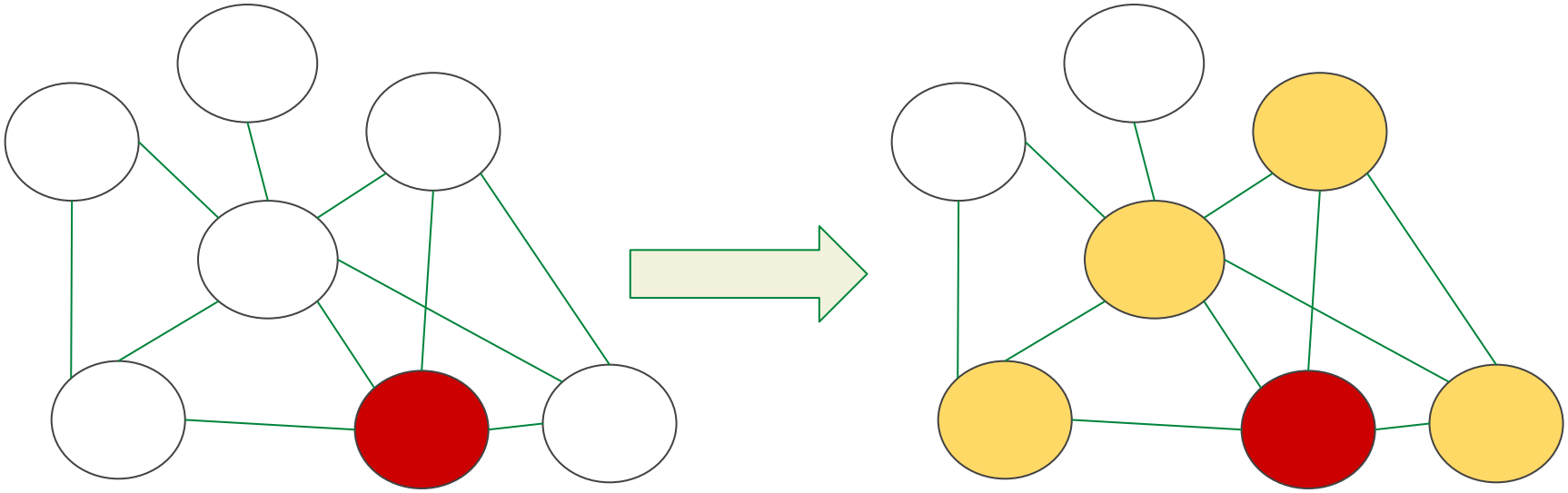


Implementación de grafos

- Matrices de adyacencia vs. Listas de adyacencia
  - En general son mejores las matrices de adyacencia si los grafos son densos ($a \approx n^2$) y las listas de adyacencia para grafos poco densos.
  - Para cada problema hay que sopesar las diferentes opciones y elegir la más adecuada.

Ejemplo: Radar COVID

- Aplicación que mantiene una lista de contactos con los que cada usuario ha mantenido un contacto estrecho
- Cuando un usuario indica que ha sido contagiado, la aplicación manda una notificación a las personas que constan como contactos estrechos



Ejercicios de grafos

1.1 Diseña una función que, dada una cadena de texto, compruebe si los paréntesis que aparecen están bien balanceados

Ejercicios de grafos

1.2 Dado este código, ¿cuál de las siguiente secuencias es una salida válida?

```
n=3;
p=nuevaPila();
for (i=1;i<=n;i++)
  if (aleatorio(i))
     printf("%d",i);
  else apilar(&p,i);
while (!esVacia(p)){
  i=cima(p);
  desapilar(&p);
  printf("%d",i);
}
```

Ejercicios de grafos

1.3 Sea G = N,A un grafo ponderado dirigido con 20 vértices. Suponemos que un elemento de una matriz ocupa una unidad de espacio; y un campo de una lista ocupa 3 unidades (una para almacenar el nodo destino, otra para el peso y otra para el puntero). ¿Hasta cuántas aristas puede tener el grafo para que ocupe menos espacio almacenado en una lista de adyacencia que en una matriz de adyacencia?

Ejercicios de grafos

E1.4. Para el mismo grafo. Suponemos que se tarda lo mismo en recorrer un elemento de la matriz que un elemento de la lista; y queremos encontrar los arcos que tienen como nodo destino el vértice 5. ¿Cuántas aristas tiene que tener el grafo para que sea más rápido hacer la búsqueda en una lista de adyacencia que en una matriz de adyacencia?

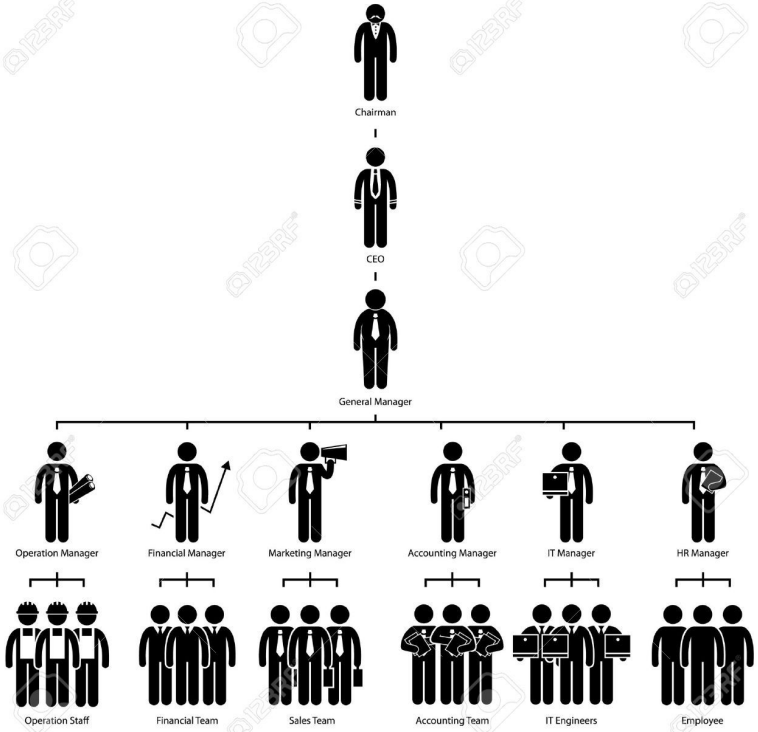
Ejercicios de grafos

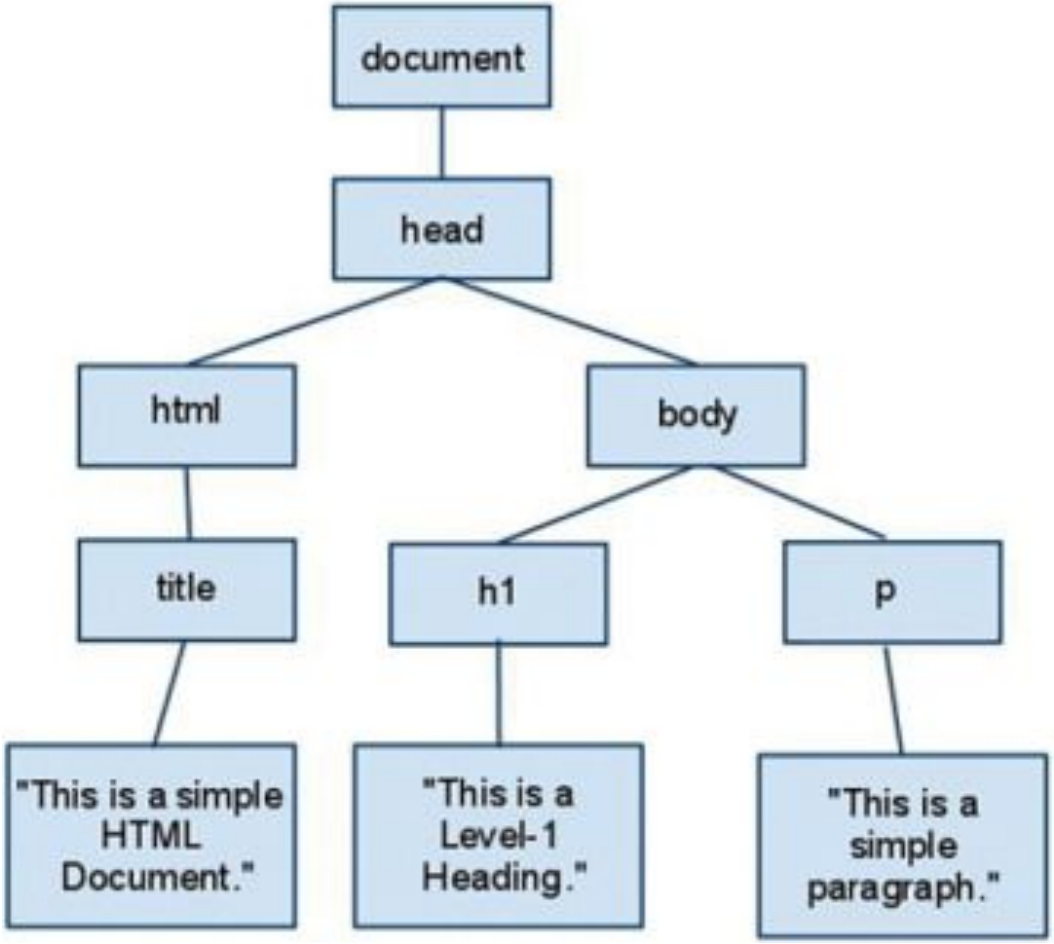
1.5 Un grafo no dirigido es euleriano si existe un ciclo (camino cerrado) euleriano que incluye a todas sus aristas. Escribir una función que, dado un grafo no dirigido, determine si es euleriano

Índice

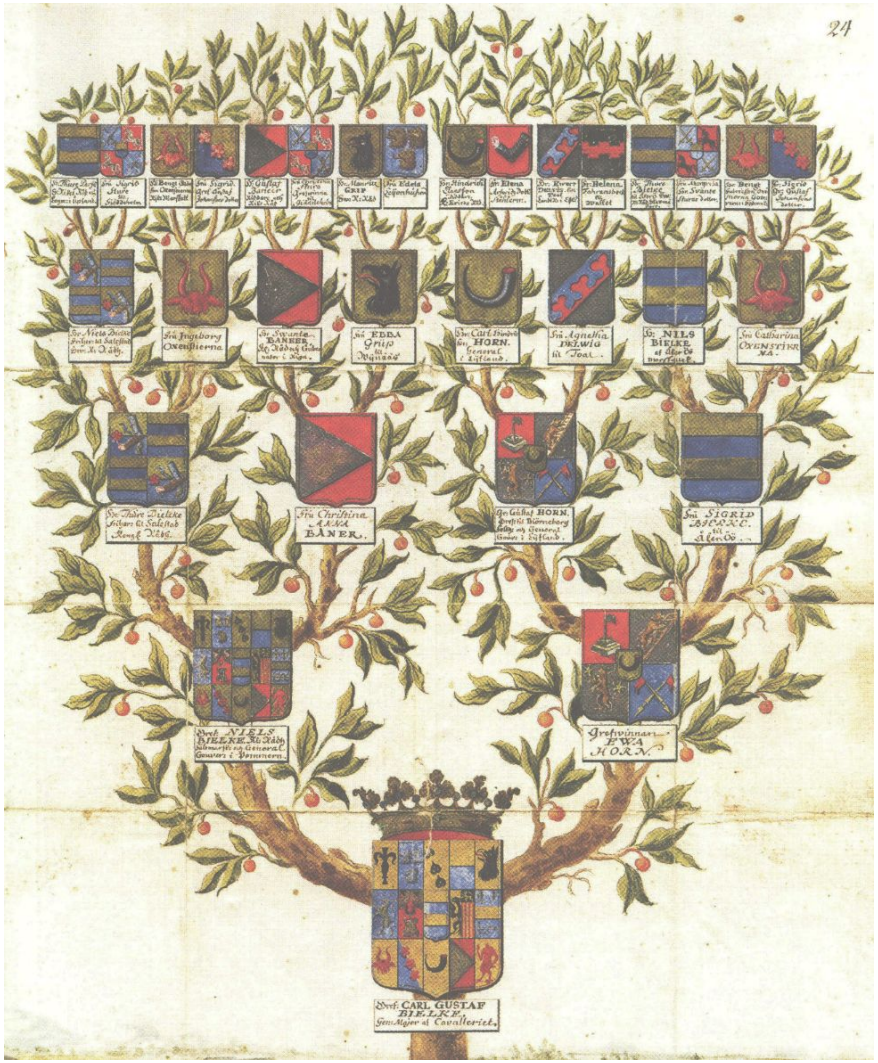
- Repaso. Estructuras de datos básicas
- Implementación de pilas
- Grafos
  - Introducción a grafos
  - Implementación de grafos
- Árboles
  - Recorridos en anchura y profundidad
  - Árboles binarios de búsqueda
  - Implementación de árboles binarios de búsqueda
  - Montículos

Árboles - Introducción





Árboles - Introducción



Árboles

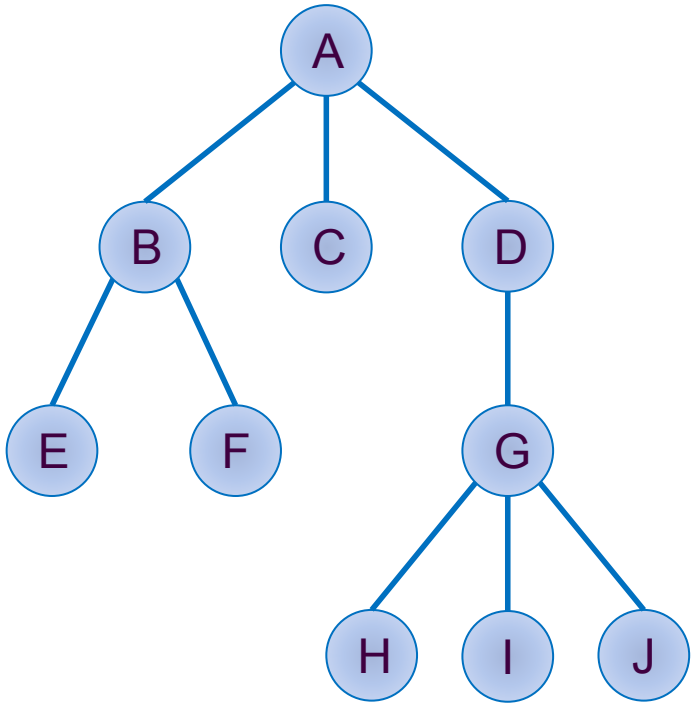
- Un árbol es un grafo no dirigido, conexo y acíclico.

- Un árbol T es un conjunto finito de uno o más nodos tal que hay un nodo especial llamado raíz y el resto están divididos en $n \geq 0$ subconjuntos disjuntos $T_1, T_2, \ldots, T_n$ de árboles a los que llamaremos subárboles de T.

- El conjunto vacío es un árbol

- La representación más habitual de un árbol es mediante listas enlazadas

Árboles



Nivel

1

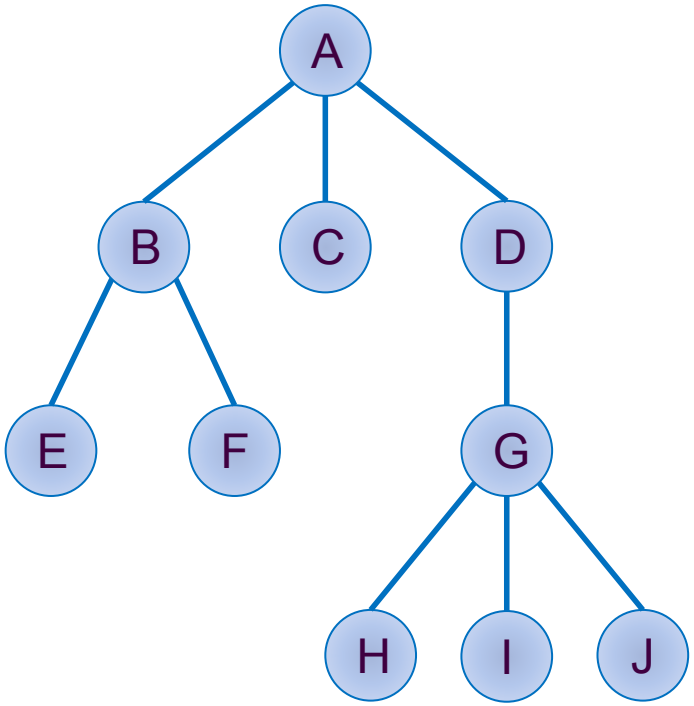
2

3

4

- Elementos de un árbol
  - **Hojas** o **nodos terminales**; **nodos no terminales**
    - E, F, C, H, I, J son nodos terminales
    - A, B, D, G son nodos no terminales
  - **Hijos**, **padres**, **hermanos**, **antecesores**, **sucesores** de un nodo
    - E es hijo de B
    - B es padre de E
    - B y D son hermanos
    - Los antecesores de J son G, D y A
    - Los sucesores de D son G, H, I y J

Árboles



- Elementos de un árbol
  - **Grado** de un nodo: número de hijos
    - A tiene grado 3
    - B tiene grado 2
    - Todos los nodos hoja tienen grado 0
  - **Nivel** de un nodo: La raíz tiene nivel 1 y, si un nodo tiene nivel p, sus hijos tienen nivel p+1
  - **Altura** o profundidad de un árbol: máximo nivel de cualquier nodo
    - El árbol del ejemplo tiene altura 4

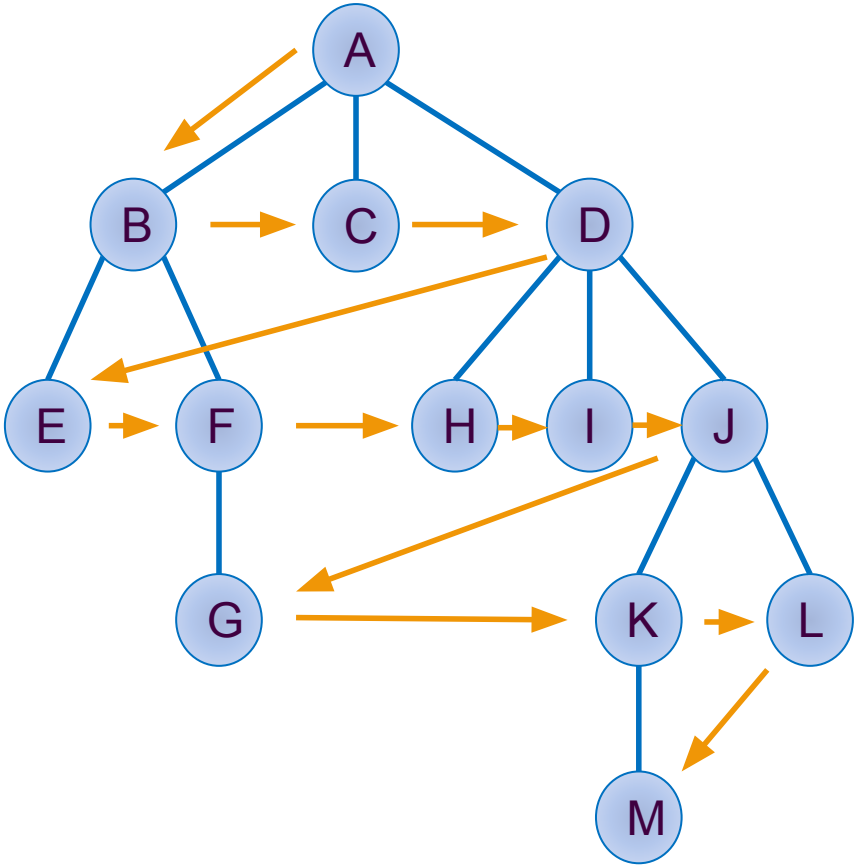
Índice

- Repaso. Estructuras de datos básicas
- Implementación de pilas
- Grafos
  - Introducción a grafos
  - Implementación de grafos
- Árboles
  - Recorridos en anchura y profundidad
  - Árboles binarios de búsqueda
  - Implementación de árboles binarios de búsqueda
  - Montículos

Recorridos en anchura y profundidad

- Muchos algoritmos requieren hacer un recorrido por los nodos de un árbol, empezando por la raíz. A veces habrá que explorar todos los vértices, a veces el recorrido terminará antes
- Dos tipos de recorridos: anchura y profundidad
- La diferencia fundamental es el orden en que se visitan los diferentes vértices.
- Para recorrer un grafo es necesario seleccionar un nodo inicial. En el caso de árboles, este nodo es la raíz del árbol.

Recorrido en anchura



- Los nodos se recorren por niveles, primero la raíz, luego los de nivel 2, luego 3, etc.

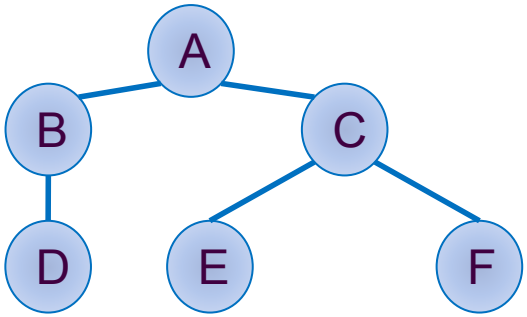
- A – B – C – D – E – F – H – I – J – G – K – L – M

Recorrido en anchura - implementación

Se puede implementar utilizando una cola:

- Al comienzo añadimos el elemento raíz a la cola
- Bucle: mientras haya elementos en la cola
  - Sacar primer elemento de la cola, procesarlo
  - Añadir los hijos de ese elemento a la cola

Recorrido en anchura



Añadimos A a la cola

Procesamos **A**, añadimos sus hijos B y C a la cola

Procesamos **B**, añadimos su hijo D a la cola

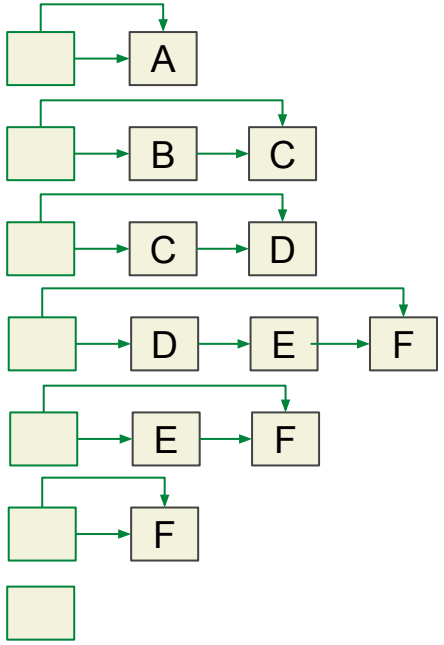
Procesamos **C**, añadimos sus hijos E y F a la cola

Procesamos **D**, no tiene hijos

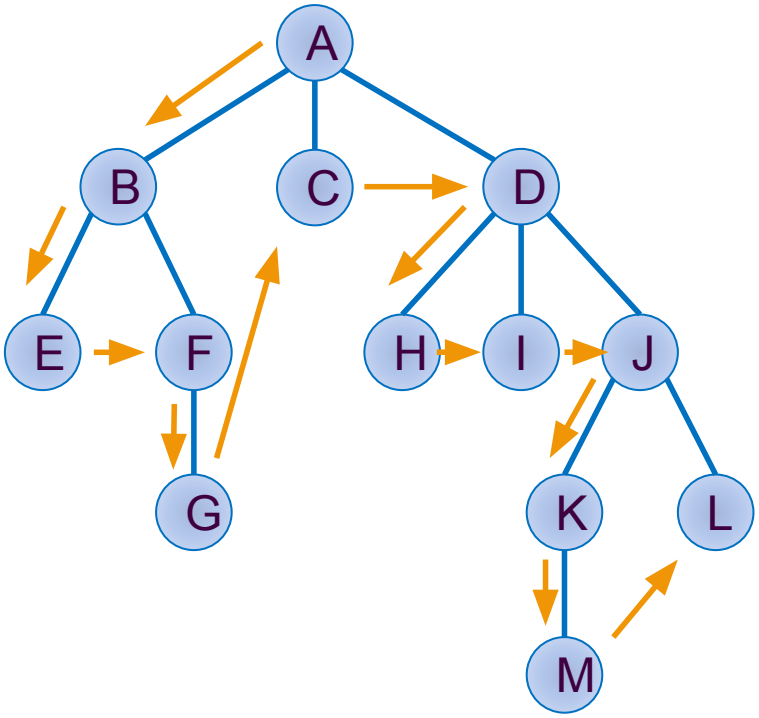
Procesamos **E**, no tiene hijos

Procesamos **F**, no tiene hijos

La cola está vacía, termina el recorrido



Recorridos en profundidad



1. Empieza en la raíz
2. va hacia abajo hasta llegar a un nodo hoja
3. vuelve hacia atrás (backtracking), hasta llegar a un nodo que tenga hijos sin explorar
4. Repite 2-3 hasta procesar todos los nodos

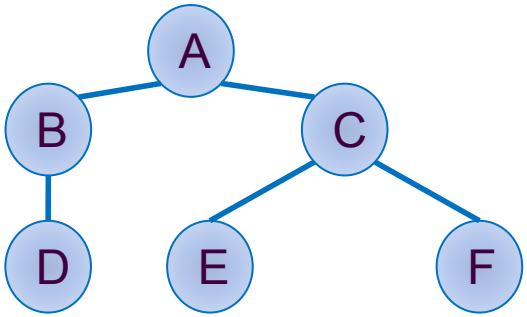
- A – B – E – F – G – C – D – H – I – J – K – M – L

Recorrido en profundidad - implementación

Se puede implementar utilizando una pila:

- Al comienzo añadimos el elemento raíz a la pila
- Bucle: mientras haya elementos en la pila
  - Sacar primer elemento de la pila, procesarlo
  - Añadir los hijos de ese elemento a la pila en orden inverso

Recorrido en profundidad



Añadimos A a la pila

Procesamos **A**, añadimos sus hijos B y C a la pila

Procesamos **B**, añadimos su hijo D a la pila

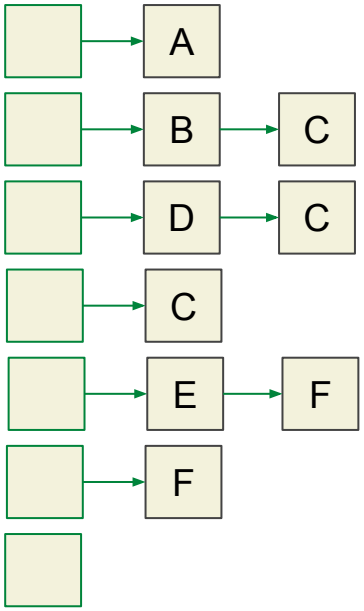
Procesamos **D**, no tiene hijos

Procesamos **C**, añadimos sus hijos E y F a la pila

Procesamos **E**, no tiene hijos

Procesamos **F**, no tiene hijos

La pila está vacía, termina el recorrido

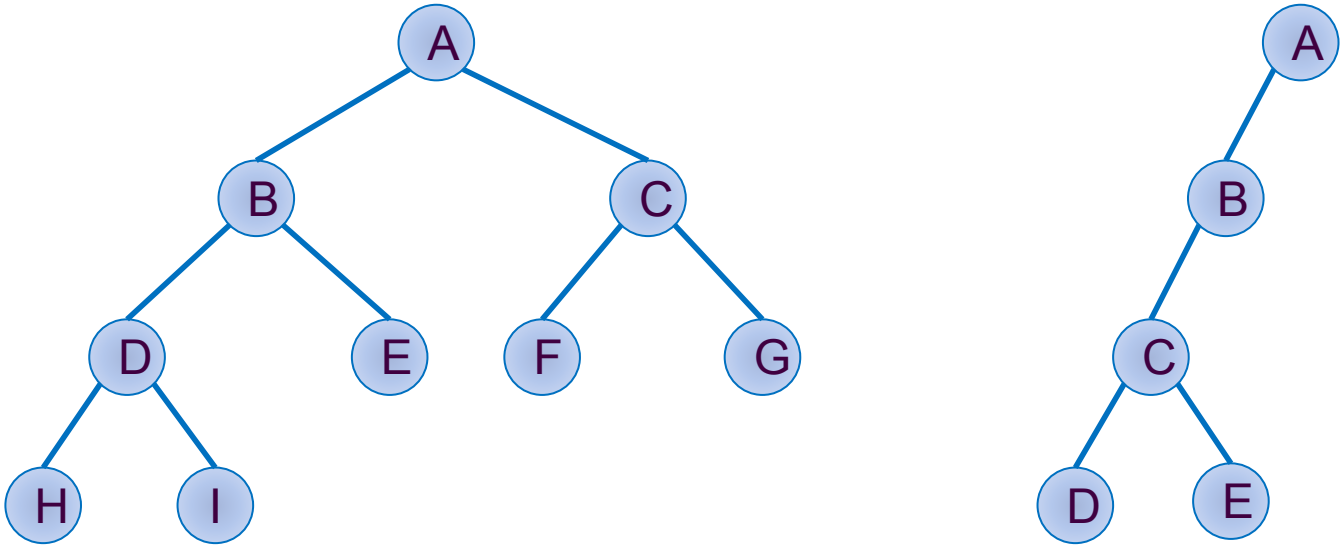


Índice

- Repaso. Estructuras de datos básicas
- Implementación de pilas
- Grafos
  - Introducción a grafos
  - Implementación de grafos
- Árboles
  - Recorridos en anchura y profundidad
  - Árboles binarios de búsqueda
  - Implementación de árboles binarios de búsqueda
  - Montículos

Árboles binarios

- Un árbol binario es un conjunto finito de nodos que puede estar vacío o formado por una raíz y dos subconjuntos disjuntos de árboles binarios llamados subárbol izquierdo y subárbol derecho

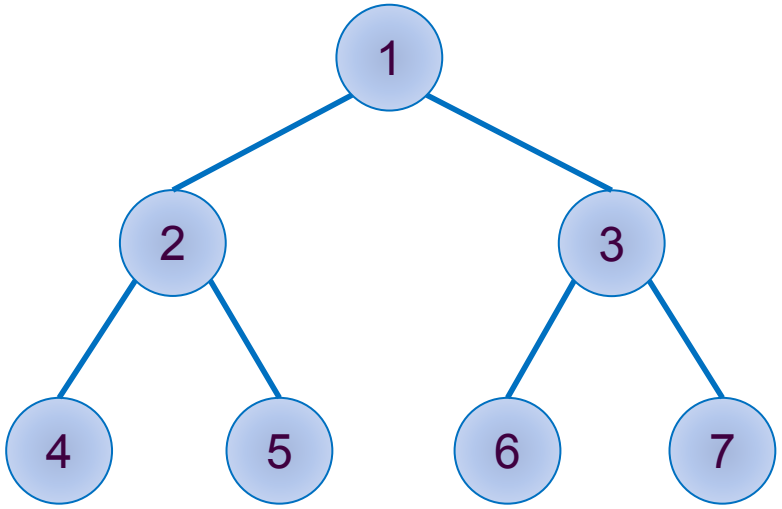


- En un árbol binario cada nodo puede tener 0, 1 o 2 hijos

Árboles binarios

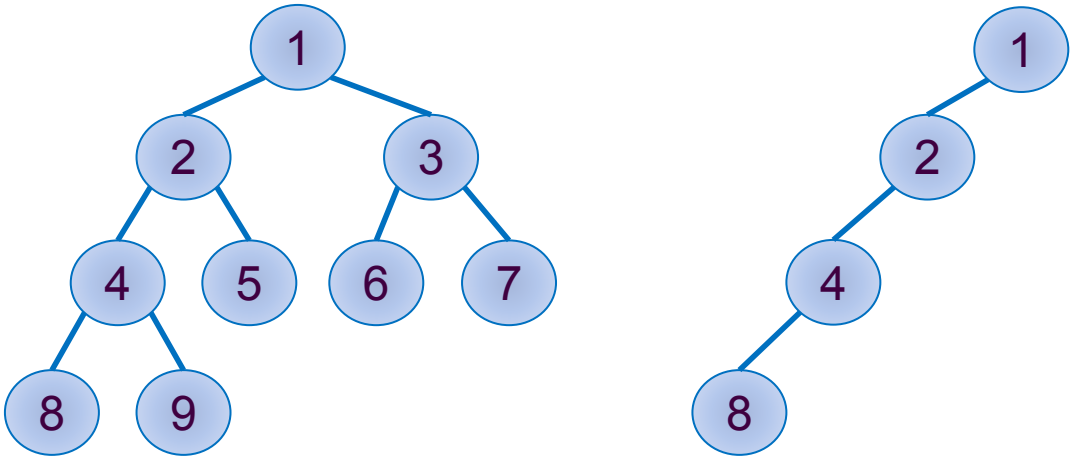
- Un árbol binario lleno de profundidad k tiene exactamente $2^k - 1$ nodos. Los numeramos de 1 a n ($n = 2^k$-1).
- Para cada cualquier nodo i (1≤i≤n):

- El padre de i es entero(i/2), si i≠1 (la raíz no tiene padre)
- El hijo izquierdo de i es 2i, si 2i≤n (en otro caso no hay hijo izquierdo)
- El hijo derecho de i es 2i+1, si 2i+1≤n (en otro caso no hay hijo derecho)



Árboles binarios

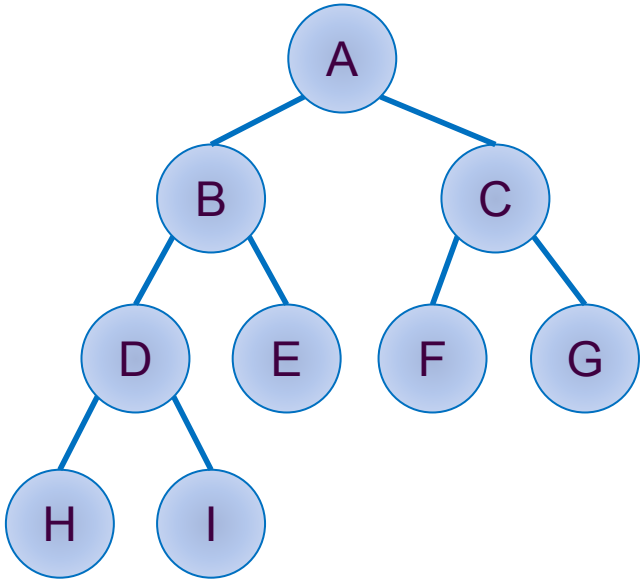
Un árbol binario de n nodos y profundidad k es completo si y sólo si sus nodos se corresponden con los nodos numerados de 1 a n en el árbol binario lleno de profundidad k.



El primero es completo (pero no lleno), el segundo no.
Todos los árboles llenos son completos
No todos los árboles completos son llenos

Árboles binarios

- Los árboles binarios completos se pueden representar mediante vectores / arrays



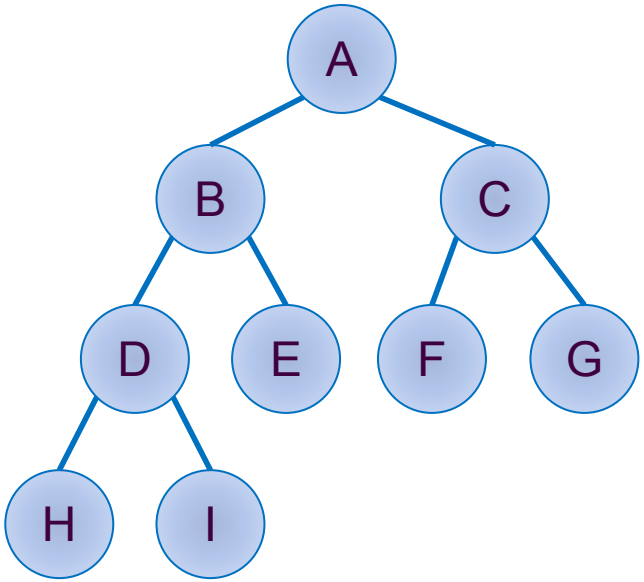


Árboles binarios

- Recorridos de un árbol binario
  - El recorrido en anchura se mantiene igual, hay varios recorridos en profundidad
  - La diferencia está en cuándo se procesa un elemento respecto a sus hijos izquierdos y derechos
  - Recorrido en pre-orden (Raíz, Izquierda, Derecha). Este recorrido es el mismo que el de profundidad para árboles no-binarios
  - Recorrido en in-orden (Izquierda, Raíz, Derecha)
  - Recorrido en post-orden (Izquierda, Derecha, Raíz)

Árboles binarios

- Recorrido en **pre-orden** (recorrido en profundidad clásico)
  - Visita la raíz
  - Recorrido del subárbol izquierdo en pre-orden
  - Recorrido del subárbol derecho en pre-orden

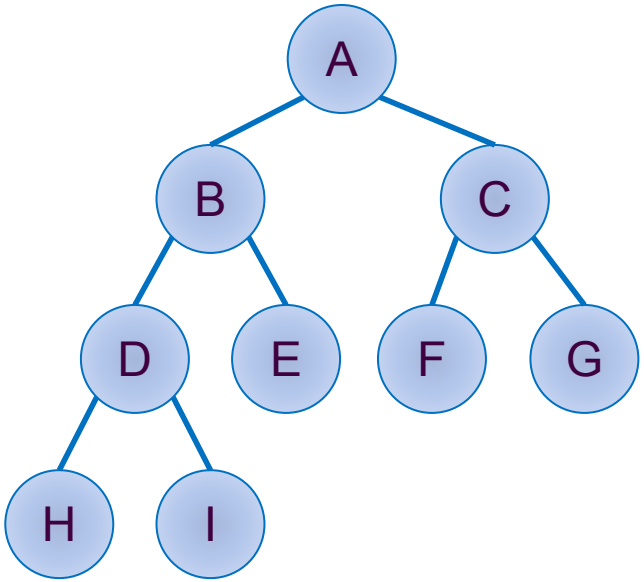



```
función preorden( Arbol a)
    si (no nulo(a))
        procesar(raiz(a))
        preorden(a->hijoIzquierdo())
        preorden(a->hijoDerecho())
    fsi
ffuncion
```

A – B – D – H – I – E – C – F - G

Árboles binarios

- Recorrido en **in-orden**
  - Recorrido del subárbol izquierdo en in-orden
  - Visita la raíz
  - Recorrido del subárbol derecho en in-orden

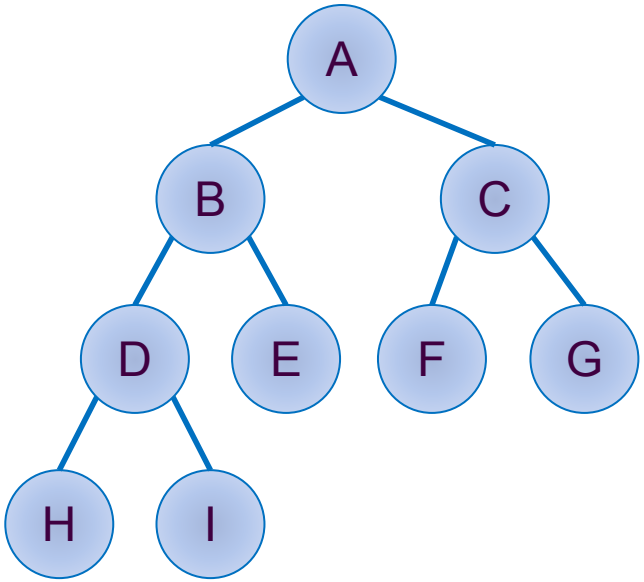



```
función inorden( Arbol a)
    si (no nulo(a))
        inorden(a->hijoIzquierdo())
        procesar(raiz(a))
        inorden(a->hijoDerecho())
    fsi
ffuncion
```

H – D – I – B – E – A – F – C - G

Árboles binarios

- Recorrido en **post-orden**
  - Recorrido del subárbol izquierdo en post-orden
  - Recorrido del subárbol derecho en post-orden
  - Visita la raíz



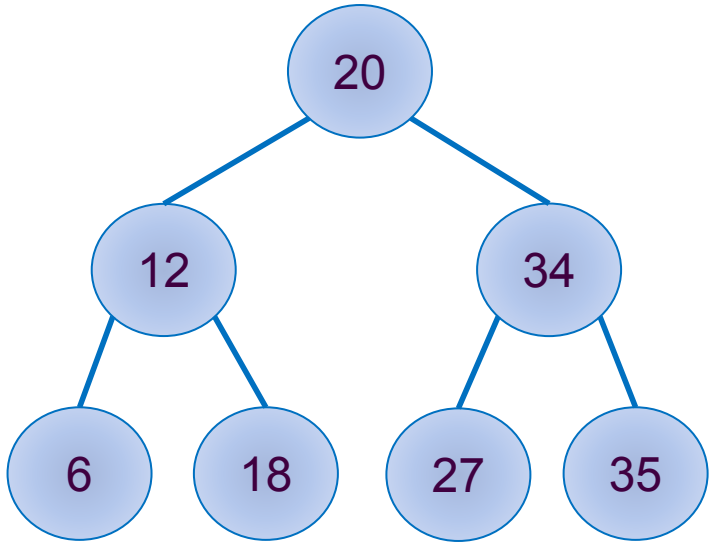

```
función postorden( Arbol a)
    si (no nulo(a))
        postorden(a->hijoIzquierdo())
        postorden(a->hijoDerecho())
        procesar(raiz(a))
    fsi
ffuncion
```

H – I – D – E – B – F – G – C - A

Árboles binarios de búsqueda

Un árbol binario de búsqueda es un árbol binario que puede ser vacío o, en caso contrario, satisface las siguientes propiedades:

- Cada elemento tiene una clave y no hay dos elementos con la misma clave
- Las claves del subárbol izquierdo son menores que la clave de la raíz
- Las claves del subárbol derecho son mayores que la clave de la raíz
- Los subárboles izquierdo y derecho son también árboles binarios de búsqueda



Árboles binarios de búsqueda

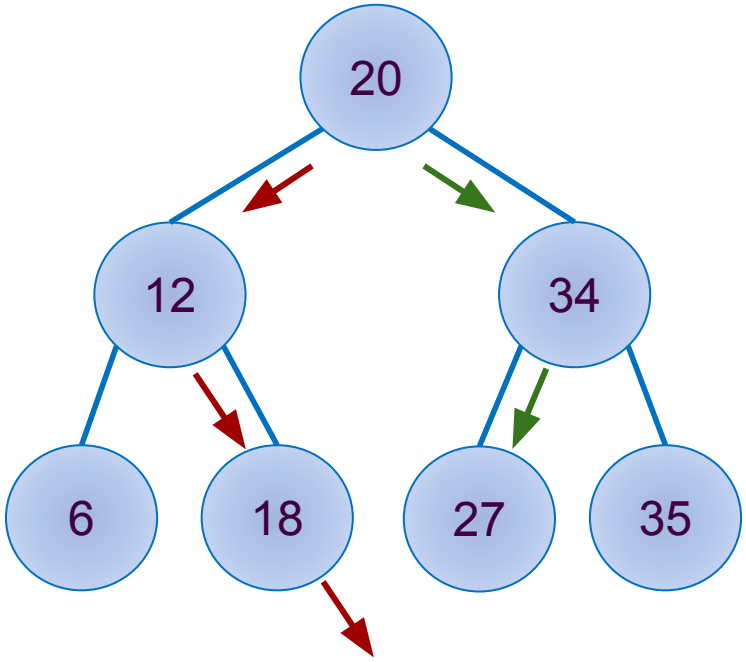
Búsqueda recursiva en un árbol binario de búsqueda

```
función busqueda(Arbol a, Entero e)
   si (nulo(a)) entonces
      devolver falso
   [] entonces
      si e = raiz(a) entonces
         devolver verdadero
      [] si e < raiz(a) entonces
         devolver busqueda(a->hijoIzquierdo, e))
      [] entonces
         devolver busqueda(a->hijoDerecho, e))
      fsi
   fsi
ffuncion
```

Como mucho se comprueban tantos nodos como la altura del árbol (log2(n) si el árbol está equilibrado)

Árboles binarios de búsqueda

Búsqueda recursiva en un árbol binario de búsqueda



Buscamos el elemento 27
Buscar (20, 27)
Buscar (34, 27)
Buscar (27, 27) ENCONTRADO

Buscamos el elemento 19
Buscar (20, 19)
Buscar (12, 19)
Buscar (18, 19)
Buscar (vacío, 19) NO ENCONTRADO

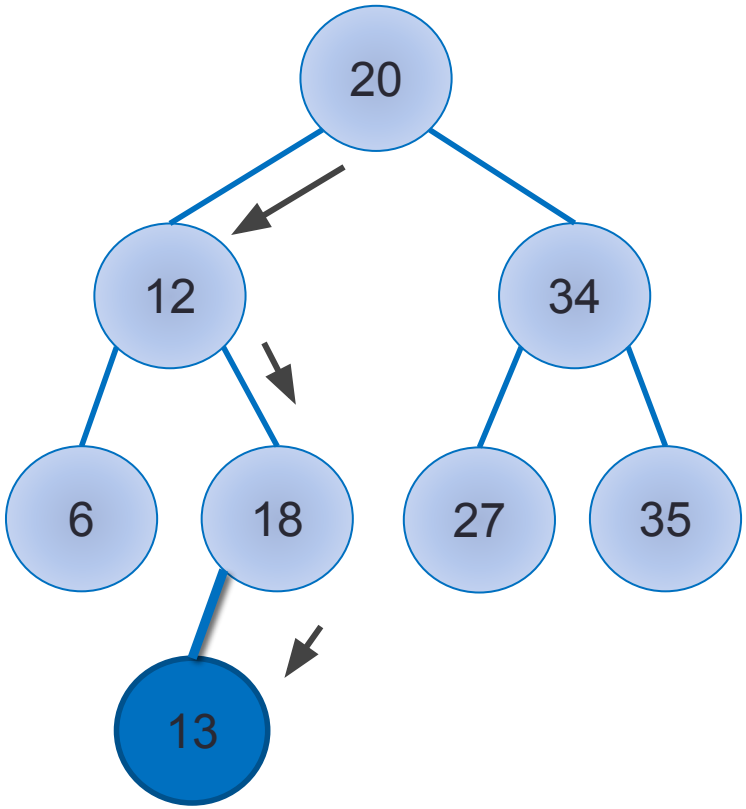
Árboles binarios de búsqueda

Inserción en un árbol binario de búsqueda

```
función insertar(Arbol a, Entero e)
   si (nulo(a)) entonces
      insertar e
   [] entonces
      si e = raiz(a) entonces
         no hacer nada
      [] si e < raiz(a) entonces
         insertar(a->hijoIzquierdo, e))
      [] entonces
         insertar(a->hijoDerecho, e))
      fsi
   fsi
ffuncion
```

Árboles binarios de búsqueda

- Inserción en un árbol binario de búsqueda



Insertar el elemento 13
Insertar (20, 13)
Insertar (12, 13)
Insertar (18, 13)
Insertar (vacío, 13) □ INSERTADO

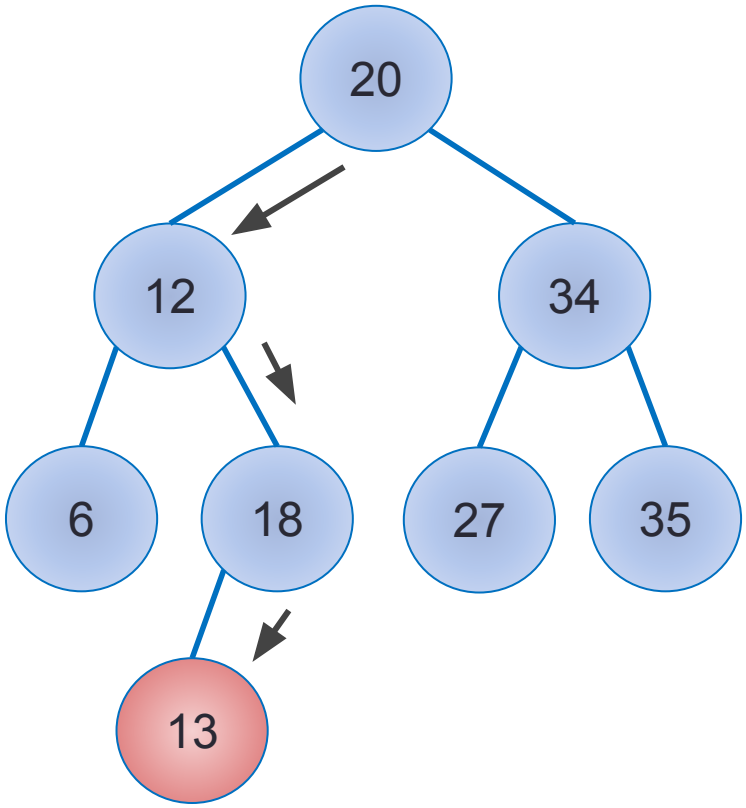
Árboles binarios de búsqueda

Borrado en un árbol binario de búsqueda

- El elemento a eliminar es una hoja
  - Simplemente elimino el nodo
- El elemento a eliminar sólo tiene un hijo
  - El padre del elemento se convierte en padre del hijo del elemento
- El elemento a eliminar tiene dos hijos
  - Busco el sucesor del elemento a eliminar (elemento más pequeño del subárbol hijo derecho)
  - Coloco el sucesor en la posición del elemento a eliminar
  - Elimino el sucesor de su posición original

Árboles binarios de búsqueda

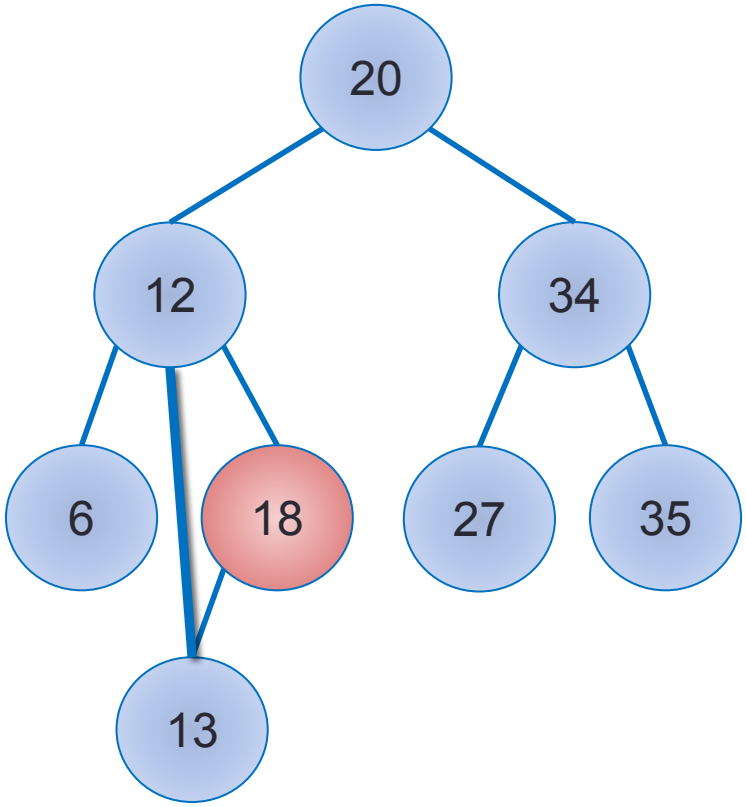
Borrado en un árbol binario de búsqueda



Borramos el elemento 13
borrar(20, 13)
borrar(12, 13)
borrar(18, 13)
borrar(13, 13)
No tiene hijos, borramos nodo 18

Árboles binarios de búsqueda

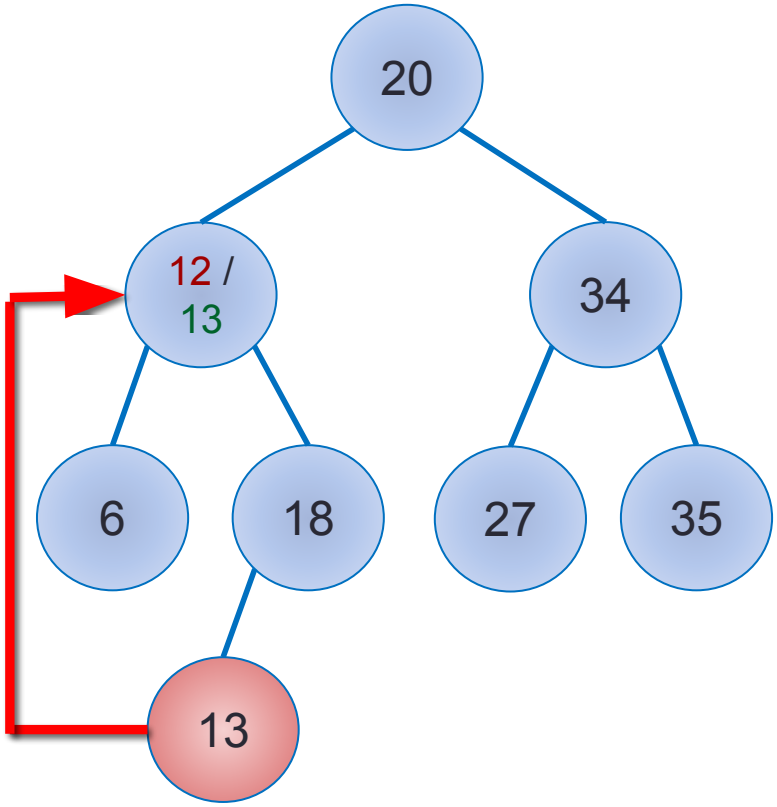
Borrado en un árbol binario de búsqueda



Borramos el elemento 18
borrar(20, 18)
borrar(12, 18)
borrar(18, 18)
Como tiene un solo hijo (13), el nuevo padre de ese hijo va a ser el padre de 18 (12)
Borramos 18

Árboles binarios de búsqueda

- Borrado en un árbol binario de búsqueda



Borramos el elemento 12

borrar(20, 12)
borrar(12, 12)
Como tiene dos hijos, buscamos a su sucesor para que lo sustituya
Su sucesor es el elemento 13
Colocar 13 en la posición de 12
Eliminar el 13 de sus posición original

Árboles binarios de búsqueda

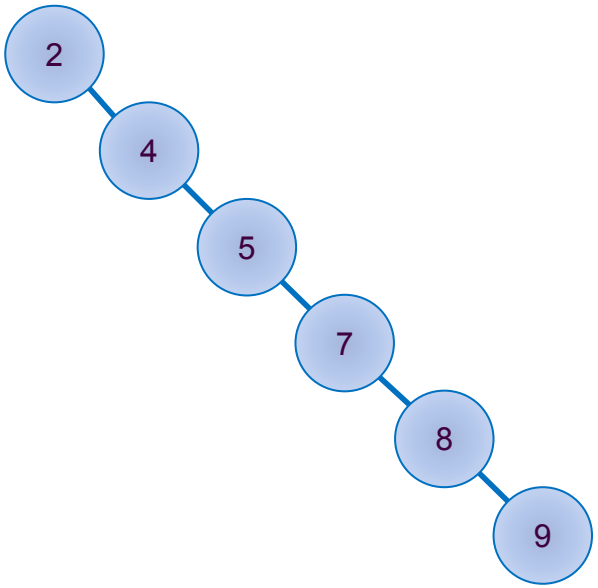
- Un mismo conjunto de datos puede crear diferentes árboles binarios de búsqueda, dependiendo del orden en que se introducen los elementos.
- Cuanto más balanceado esté el árbol, más eficiente serán las búsquedas que realicemos.

Árboles binarios de búsqueda

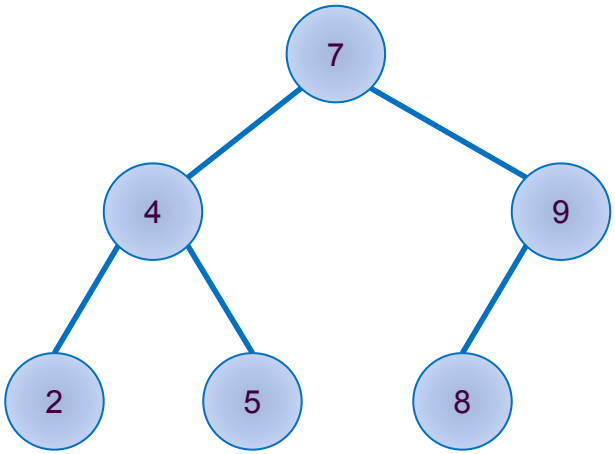
Para que un árbol esté balanceado, la raíz debe coincidir con la mediana de los elementos.

Diferentes árboles en función del orden de inserción.

{2, 4, 5, 7, 8, 9}

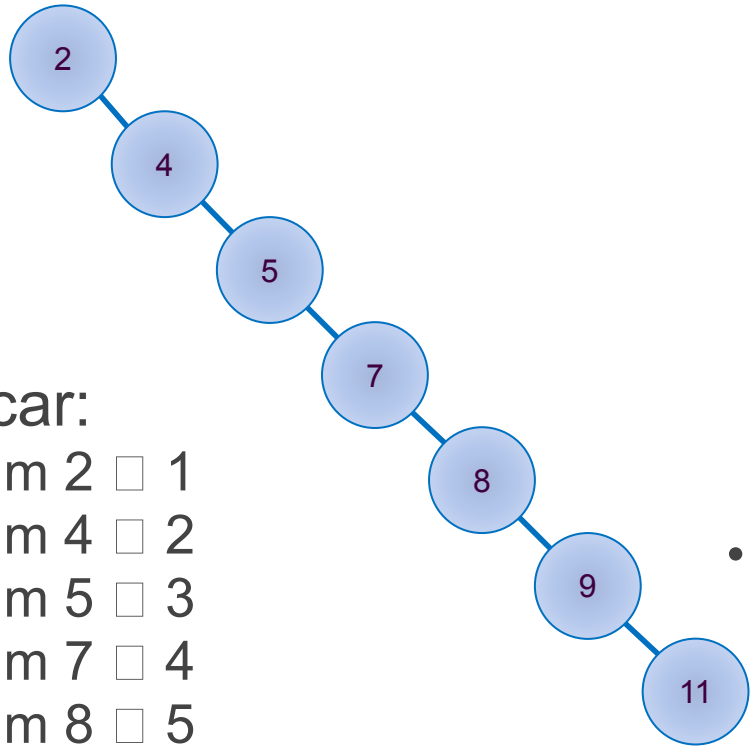


{7, 4, 9, 8, 2, 5, 11}



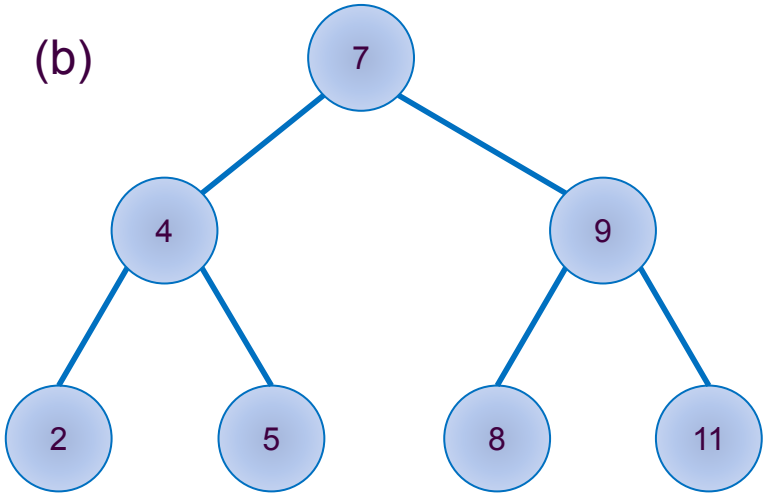
Árboles binarios de búsqueda

(a)



- Buscar:
  - Num 2 □ 1
  - Num 4 □ 2
  - Num 5 □ 3
  - Num 7 □ 4
  - Num 8 □ 5
  - Num 9 □ 6
  - Num 11 □ 7
- Promedio:
  - 4 comparaciones

(b)



- Buscar:
  - Num 2 □ 3
  - Num 4 □ 2
  - Num 5 □ 3
  - Num 7 □ 1
  - Num 8 □ 3
  - Num 9 □ 2
  - Num 11 □ 3

Promedio:
2,43 comparac.

Índice

- Repaso. Estructuras de datos básicas
- Implementación de pilas
- Grafos
  - Introducción a grafos
  - Implementación de grafos
- Árboles
  - Recorridos en anchura y profundidad
  - Árboles binarios de búsqueda
  - Implementación de árboles binarios de búsqueda
  - Montículos

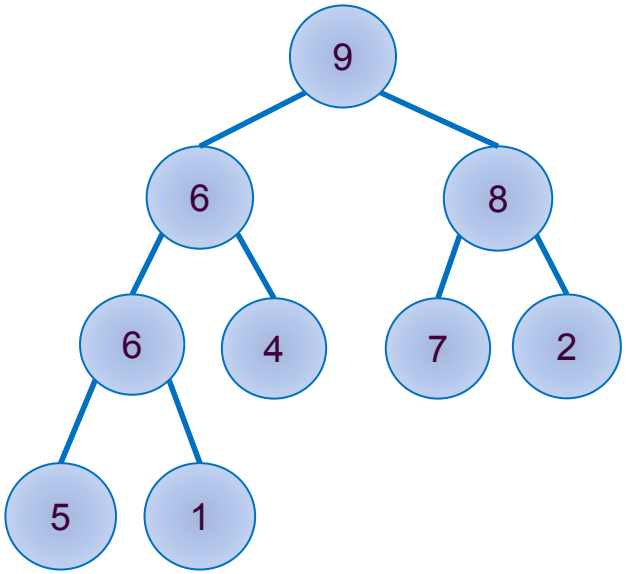
Montículos

- Un max-montículo es un árbol binario completo tal que el valor de cada nodo es mayor o igual que los valores de los nodos de sus hijos.
- Un min-montículo es un árbol binario completo tal que el valor de cada nodo es menor o igual que los valores de los nodos de sus hijos.
- La raíz de un max-montículo contendrá uno de los mayores valores (el mayor si todos son distintos) y la de un min-montículo uno de los menores valores.
- Ofrecen dos operaciones: inserción de un elemento nuevo, borrado de la raíz

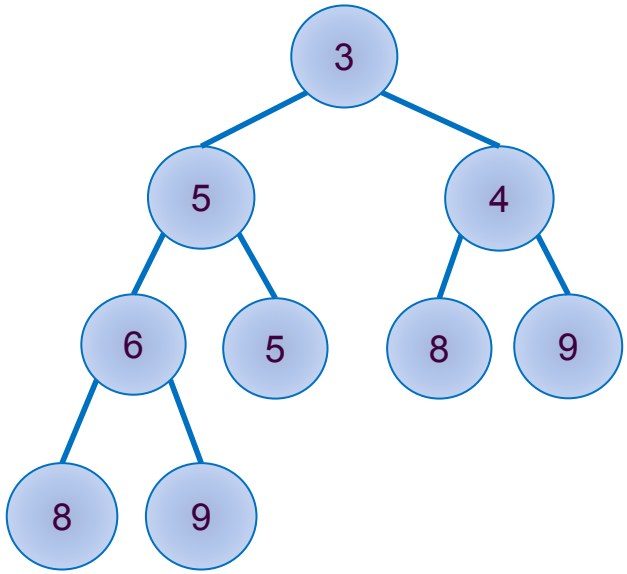
Montículos

- Un montículo, como todo árbol completo, se puede representar mediante un vector unidimensional.

max-montículo



min-montículo

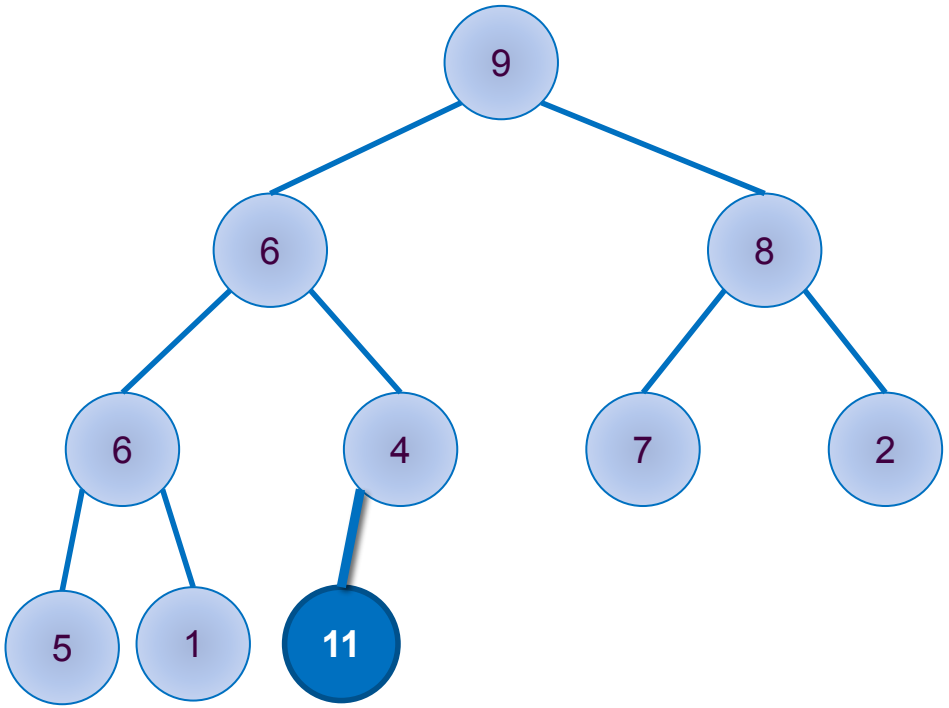






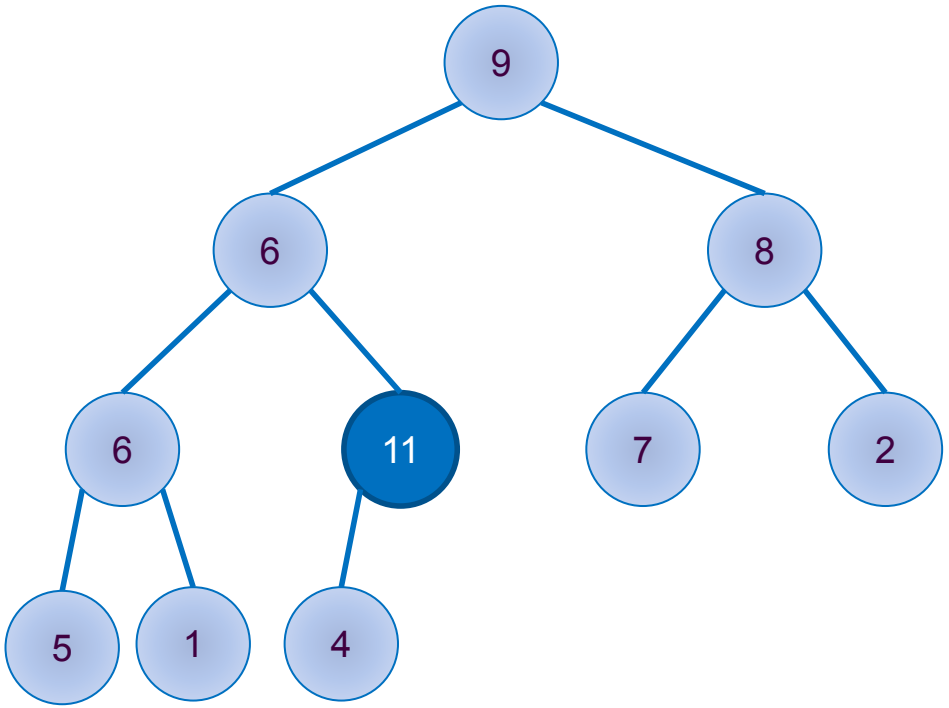
Montículos

- Inserción en un montículo
  - Insertamos el elemento en la siguiente posición
  - Mientras el nuevo elemento sea mayor que su padre (max-montículo) y no esté en la raíz, intercambiar elemento por su padre



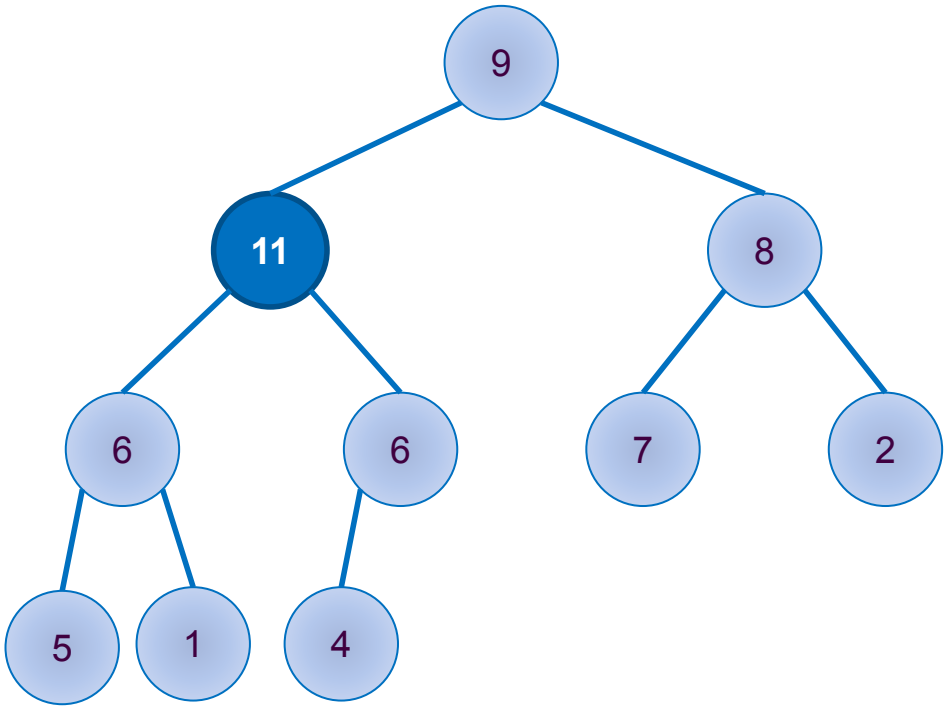
Montículos

- Inserción en un montículo
  - Insertamos el elemento en la siguiente posición
  - Mientras el nuevo elemento sea mayor que su padre (max-montículo) y no esté en la raíz, intercambiar elemento por su padre



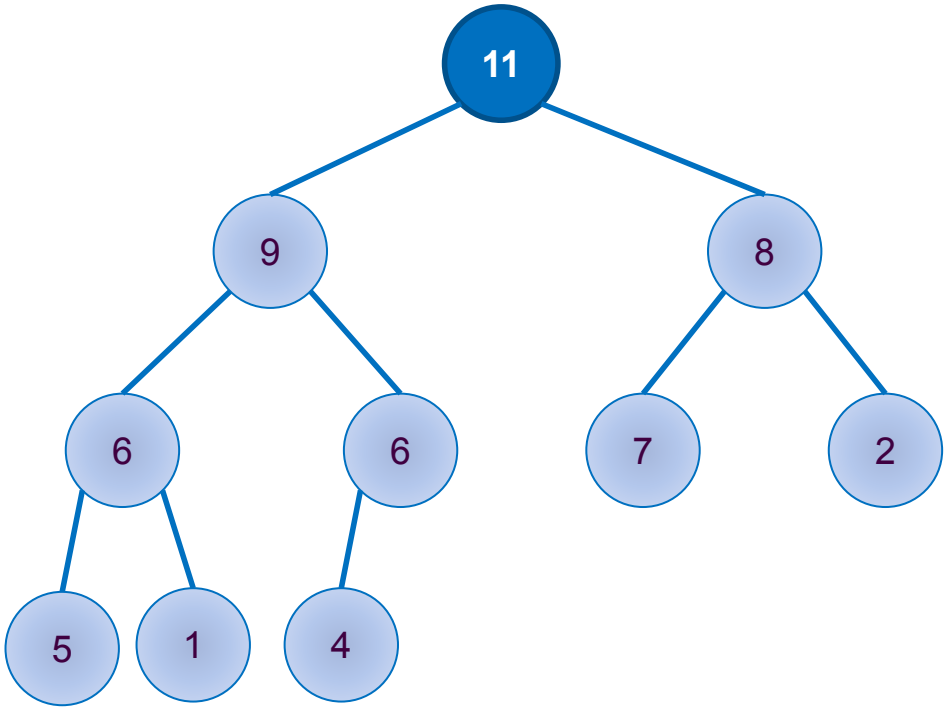
Montículos

- Inserción en un montículo
  - Insertamos el elemento en la siguiente posición
  - Mientras el nuevo elemento sea mayor que su padre (max-montículo) y no esté en la raíz, intercambiar elemento por su padre



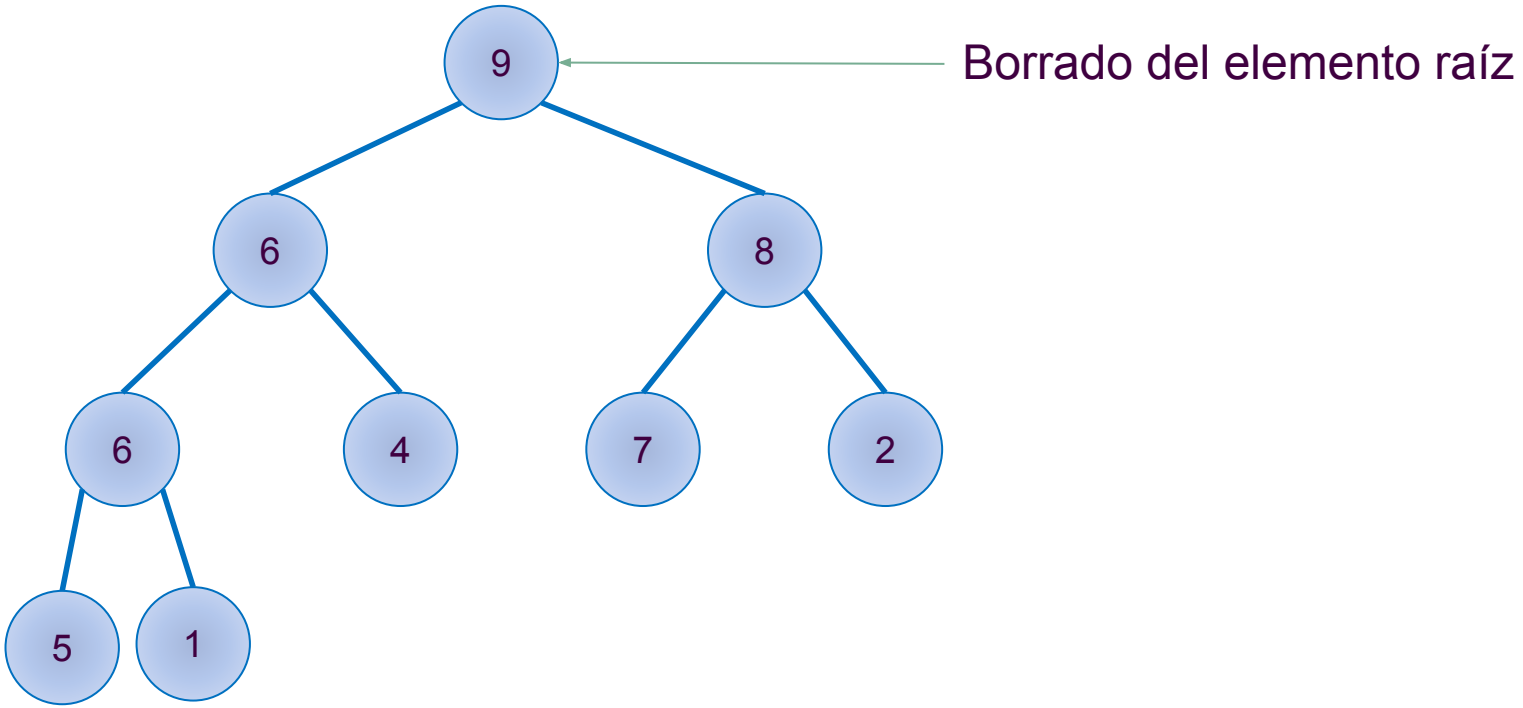
Montículos

- Inserción en un montículo
  - Insertamos el elemento en la siguiente posición
  - Mientras el nuevo elemento sea mayor que su padre (max-montículo) y no esté en la raíz, intercambiar elemento por su padre



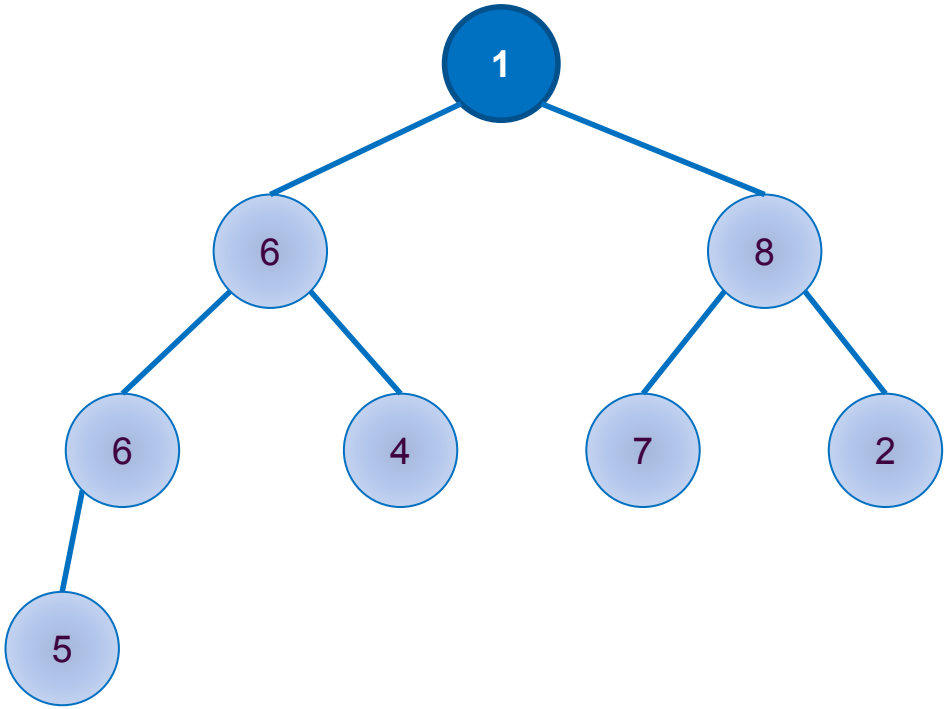
Montículos

- Borrado del elemento raíz en un montículo
  - Borramos el elemento y colocamos el último elemento en su posición
  - Mientras el elemento movido sea menor que alguno de sus hijos (max-montículo), intercambiar elemento por su hijo mayor



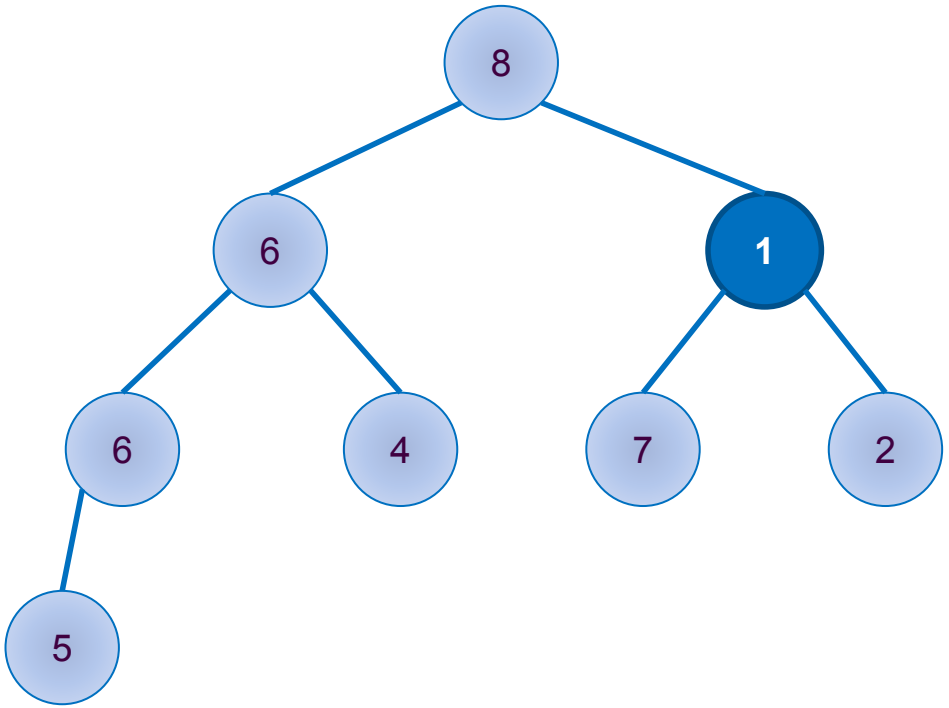
Montículos

- Borrado del elemento raíz en un montículo
  - Borramos el elemento y colocamos el último elemento en su posición
  - Mientras el elemento movido sea menor que alguno de sus hijos (max-montículo), intercambiar elemento por su hijo mayor



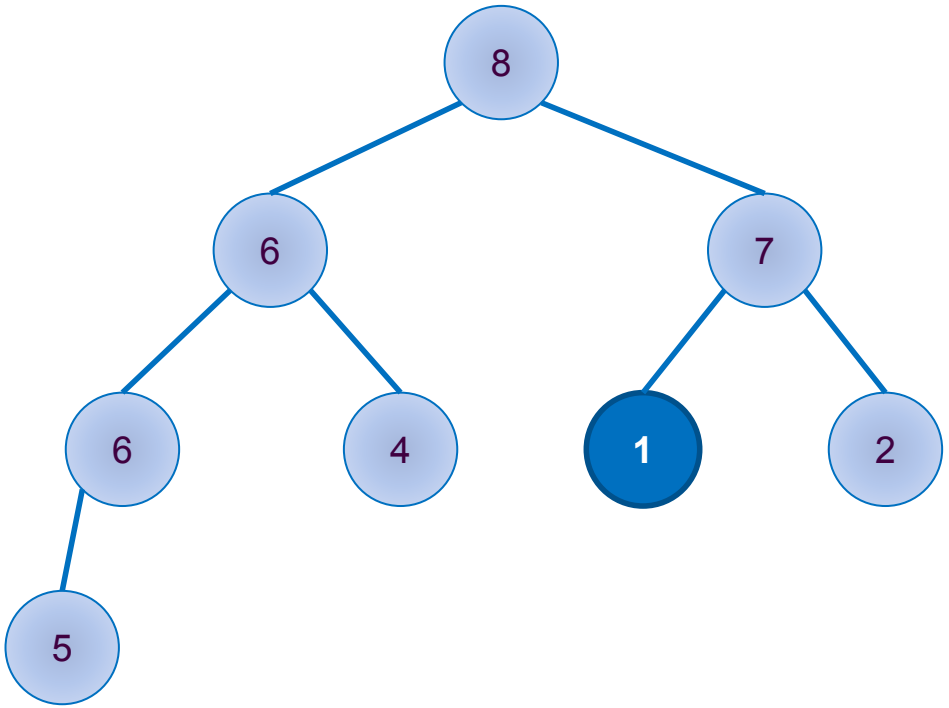
Montículos

- Borrado del elemento raíz en un montículo
  - Borramos el elemento y colocamos el último elemento en su posición
  - Mientras el elemento movido sea menor que alguno de sus hijos (max-montículo), intercambiar elemento por su hijo mayor



Montículos

- Borrado del elemento raíz en un montículo
  - Borramos el elemento y colocamos el último elemento en su posición
  - Mientras el elemento movido sea menor que alguno de sus hijos (max-montículo), intercambiar elemento por su hijo mayor



Montículos

- Útiles cuando sólo interesa conocer el elemento con mayor (en max-montículos) o menor (en min-montículos) valor
- En la primera posición está el elemento mayor o menos, no se garantiza el orden relativo entre el resto de posiciones
- Las operaciones para añadir y borrar elementos son muy eficientes (log2(n))
- Ejemplo de aplicación: colas de prioridad, estructura de datos optimizada para devolver el elemento que tenga mayor o menor prioridad

Ejercicios disponibles

El resto del tema 1